Concretely-Efficient Multi-Key Homomorphic Secret Sharing and Applications

Kaiwen He
MIT CSAIL
khe01@mit.edu

Sacha Servan-Schreiber
Tinfoil
sacha@tinfoil.sh

Geoffroy Couteau
Université Paris Cité, CNRS, IRIF
couteau@irif.fr

Srinivas Devadas
MIT CSAIL
devadas@mit.edu

***Abstract*—Homomorphic secret sharing (HSS) is a powerful cryptographic primitive that enables efficient, low-communication secure computation without the use of fully homomorphic encryption. Public-key HSS is a well-known variant that supports inputs from multiple parties, but all parties must agree on a joint public key *before* any party can encode their inputs, requiring extra rounds of communication in applications. Recently, Couteau et al. (EUROCRYPT 2025) constructed multi-key HSS (MKHSS)—a new primitive which allows parties to encode their inputs under independent keys—under the DCR assumption. MKHSS assumes only a reusable common reference string, without the need for prior interactions between parties or a public-key infrastructure.**

In this paper, we construct and implement the first concretely-efficient MKHSS scheme under the same assumptions used by Couteau et al. Using an algorithmic insight that reduces the largest modulus in Couteau et al. from N^4 to N^2, our optimized implementation can homomorphically multiply inputs in 5.0 milliseconds—while an implementation of Couteau et al. requires 224.6 milliseconds—thereby achieving a $45\times$ speedup.

A powerful application of MKHSS is to realize attribute-based non-interactive key exchange (ANIKE), which generalizes password-authenticated key exchange (PAKE) to arbitrary attribute policies. ANIKE is currently only known from MKHSS.

We use our implementation to evaluate the first concretely-efficient ANIKE schemes for a range of practically useful policies. Using our implementation, two parties can perform a geolocation-based key exchange in under one second and a fuzzy PAKE on an 8-word passphrase in a few seconds for realistic parameters, on a single core, achieving a roughly $30\times$ speedup over Couteau et al. for both applications.

1. Introduction

Homomorphic secret sharing (HSS) [1] allows two or more parties, each holding a public encoding of some secret input x, to locally compute a function f on the input and obtain a subtractive secret share of $f(x)$. It provides an efficient, distributed alternative to fully-homomorphic encryption (FHE) [2] for secure computation.

The most basic form of HSS only supports computation over private input provided by a single party. The well-known variant, called public-key HSS [3], [4], [5], supports inputs by multiple parties. However, all public-key HSS schemes require that all parties agree on some joint public key *before* they can even generate the input encodings, which are typically ciphertexts encrypted under the joint key. In addition, the parties who wish to evaluate a function need to obtain correlated evaluation keys, which can be seen as additive shares of the joint secret key. This requirement often requires an extra round of communication in many applications.

To address these shortcomings, Couteau et al. [6] introduced a new primitive called multi-key HSS (MKHSS): using only a common reference string (CRS), each party can generate their own public and secret key pair, *and* generate input encodings under its own key, without interaction with the other party. Each party can then exchange the concatenation of their public key and input encodings in a simultaneous message, and finally locally compute a common function.

We stress that the CRS can be generated once and used in multiple instances of the MKHSS protocol. Furthermore, the public keys and input encodings can both be reused across multiple executions of the protocol, potentially between *different* parties. The reusability of the CRS and public encodings is a significant advantage of MKHSS over public-key HSS, where the correlated evaluation keys are tied to the joint secret key and not reusable. The CRS can even be generated using a secure multi-party computation protocol, which will remove the trust on a single CRS generator, while maintaining protocol efficiency since this only need to be done once.

While the notion of MKHSS is promising, Couteau et al. [6] only provided a *theoretical* construction for MKHSS. It performs poorly in practice due to the need to use a modulus that is *four* times as large as an RSA modulus for the same level of security. We uncover algorithmic insights and overcome this major obstacle. In the end, our final construction only requires a modulus of *half* the size, matching the modulus size of the Paillier cryptosystem [7], a simple cryptographic construction from the DCR assumption.

The reduced modulus size allows our scheme to achieve a significant concrete improvement in the running time of every procedure as well as savings in communication and local storage. To give an example, a single homomorphic multiplication (the bottleneck in the key exchange applica-

2026 IEEE Symposium on Security and Privacy (SP) | 979-8-3315-6065-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/SP63933.2026.00004

© 2026, Kaiwen He. Under license to IEEE.

DOI 10.1109/SP63933.2026.00004
Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

tions) takes 5.0 milliseconds, a $45\times$ speedup compared to 224.6 milliseconds obtained by our implementation of the theoretical construction of Couteau et al.

1.1. An Application of MKHSS

Attribute-based non-interactive key exchange (ANIKE) is a new primitive that allows two parties—each holding some private input—to agree on a common key if and only if their inputs satisfy some policy predicate, and does so without revealing to each other any information beyond whether the predicate is satisfied.

Moreover, ANIKE has the same communication pattern as a non-interactive key exchange (NIKE) [8], e.g., Diffie-Hellman [9]. This property distinguishes it from all prior works on interactive key exchange [10], [11], [12] for practically interesting policies.

ANIKE generalizes the notion of PAKE [13] and even fuzzy PAKE [10], [12]. Couteau et al. [6] provided the first *theoretical* construction of ANIKE for all HSS-computable policies by using MKHSS as a black box.[1]

However, their construction is theoretical for more than one reason. Even if the underlying MKHSS scheme is made practical, they require evaluating a pseudorandom function (PRF) *homomorphically* over the encoded inputs, which is prohibitively expensive. We provide a practical alternative construction that only requires hashing *outside of* HSS. Our implementation exploits a property specific to our optimized scheme (and Couteau et al.) to unlock performance gains, but can still be made concretely-efficient with black-box use of a generic MKHSS construction.

We show that our ANIKE can realize predicates with concrete applications: geolocation-based key exchange and fuzzy password-authenticated key exchange (PAKE).

Importance of Non-interactivity. Public-key cryptography is predicated on the existence of non-interactivity: parties can post their keys on the internet and anyone can non-interactively encrypt a message under that key. In contrast, alternative MPC based approaches to key exchange [10], [11], [12] break this model since both parties need to be *simultaneously online* to derive a common key, which is incompatible with the modern client-server setting [14].

1.2. Our Contributions

To summarize, our contributions are:

1) Several insightful algorithmic optimizations to MKHSS that critically improve performance.
2) Identifying two concretely-practical and useful applications of MKHSS: geolocation-based key exchange and fuzzy PAKE based on passphrases.
3) A concretely-efficient, open-source C++ implementation of multi-key HSS, a recent primitive, as well as its two applications to key exchange.[2]

1. Existing HSS schemes are sufficiently powerful to compute polynomial-size branching programs, which capture all functions in NC^1.

2. Open-source code is available at: https://github.com/kevin-he-01/mkhss/.

2. Technical Overview: MKHSS

A homomorphic secret sharing (HSS) scheme typically supports a computation model known as *restricted multiplication straight-line* (RMS) programs, which homomorphically operates on B-bounded integer values of two types: input and memory values. A program starts with an initial memory value of 1 and a list of input values, and is capable of performing the following operations:

1) Linear operation on values of the same type.
2) Multiplication between an input value and a memory value, producing a memory value (RMS multiplication).

The starting point of our construction is the construction of MKHSS from the decisional composite residuosity (DCR) assumption [6]. To achieve practical performance gains, we rely on an assumption called the short-exponent indistinguishability (SEI) assumption [15], [16], which is assumed by Couteau et al. [6] when providing concrete running time estimates for MKHSS. Over a cyclic group $\mathbb{G}$ generated by g, SEI says that it is infeasible to distinguish between (g, g') and (g, g^r), where $g' \leftarrow\!\$\ \mathbb{G}$ is a random group element, and $r \leftarrow\!\$\ \{0, \ldots, 2^{2\lambda}\}$ is the *short* exponent.

Security of SEI. It was proven in [16, Appendix A] that SEI is equivalent to the standard short-exponent discrete logarithm (SEDL) assumption, which says that it is infeasible to *find* the short exponent $r \in \{0, \ldots, 2^{2\lambda}\}$ given (g, g^r). Pollard [17] introduced SEDL in 1978 (merely two years after Diffie-Hellman [9]) and provided Pollard's lambda algorithm, which remains the best cryptanalysis to our knowledge, and justifies our exponent bound of $2^{2\lambda}$ for security parameter λ. Since then, many prior works (e.g., [18], [19]) on applied and theoretical cryptography assume SEI or SEDL.

In addition to being a decades-old, well-established assumption, SEI and SEDL are often assumed in practice. Internet standards for TLS [20] and SSH [21] both suggest the use of short exponents for performance reasons. A 2016 study [22] reports that eight out of nine popular cryptographic libraries (most notably OpenSSL) *allow* the use of short exponents, and *deployed* applications (e.g., Lighttpd and Unbound) use short exponents.

2.1. Notation

We let $a \wedge b$ and $a \vee b$ denote logical conjunction (AND) and disjunction (OR), respectively. For a logical proposition P, e.g., $x < y$, define $\mathbb{1}(P) = 1$ if P is true and 0 otherwise.

Denote $[m] := \{0, 1, \ldots, m-1\}$, $\mathbb{Z}_m$ as the additive group of integers modulo m, and $\mathbb{Z}_m^*$ as the multiplicative group of integers coprime with m modulo m. A boldface variable $\mathbf{x} := (x_1, x_2, \ldots, x_L)$ denotes a vector, and an uppercase boldface letter $\mathbf{A}$ denotes a matrix or higher-dimensional tensor.$\langle y \rangle = (\langle y \rangle_A, \langle y \rangle_B)$ denotes a subtractive secret share of y between two parties (called Alice and Bob), such that $\langle y \rangle_A - \langle y \rangle_B = y$ over the *integers*. The notation extends to vectors and matrices by element-wise

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

secret sharing. $\sigma \in \{A, B\}$ denotes the identity of some arbitrary party, and $1 - \sigma$ denotes the *other* party that is not σ. We let s denote the secret key of Paillier-ElGamal encryption [23], which we use to instantiate HSS.

2.2. Prior Work

2.2.1. HSS from Paillier-ElGamal. We describe a key idea to achieve single-key two-party HSS for RMS programs based on Paillier-ElGamal encryption [23]. This is similar to [24], but simplified using short exponents.

We first show how to secret share memory and input values in an RMS program:

1) *Memory share*: Denoted $\langle\!\langle x \rangle\!\rangle_\sigma$. It is defined as $\langle\!\langle x \rangle\!\rangle_\sigma = (\langle x \cdot s \rangle_\sigma, \langle x \rangle_\sigma)$, a pair of subtractive shares.
2) *Input share*: Denoted $[\![x]\!]_\sigma$. It is generated by sampling $r, r' \leftarrow\!\$\, [2^{2\lambda}]$ and computing

$$[\![x]\!]_\sigma = ((\mathsf{ct}_0, \mathsf{ct}_1), (\mathsf{ct}'_0, \mathsf{ct}'_1)) = ((g^r(1+N)^x, g^{-rs}), (g^{r'}, g^{-r's}(1+N)^x)).$$

All Paillier-based [7] schemes use the observation that $(1+N)^x \equiv 1 + Nx \bmod N^2$ due to the binomial theorem, and therefore, the base-$(1+N)$ discrete log of $z \in \mathbb{Z}^*_{N^2}$ is $(z-1)/N$. A ciphertext in Paillier-ElGamal is a pair of $\mathbb{Z}^*_{N^2}$ elements (c_0, c_1). To decrypt, we compute $\mathsf{DLog}_{1+N}(c_0^s c_1) = (c_0^s c_1 - 1)/N$. Regarding the components of $[\![x]\!]_\sigma$, one may verify that $(\mathsf{ct}_0, \mathsf{ct}_1)$ decrypts to xs and $(\mathsf{ct}'_0, \mathsf{ct}'_1)$ decrypts to x.

Using the linearity of subtractive shares and additive homomorphism of Paillier-ElGamal, it is trivial to perform addition, subtraction, and scalar multiplications on shares of the same type. It remains to show how to do the RMS multiplication: obtain $\langle\!\langle xy \rangle\!\rangle_\sigma$, given $\langle\!\langle y \rangle\!\rangle_\sigma$ and $[\![x]\!]_\sigma$.

Let's assume for now that somehow we have y and s in the clear, so we can transform $(\mathsf{ct}_0, \mathsf{ct}_1)$—an encryption of x—into an encryption of xy via component-wise exponentiation by y (using additive homomorphism of Paillier-ElGamal), and then decrypt using s. When written down, this looks like $\mathsf{DLog}_{N+1}(\mathsf{ct}_0^{ys} \cdot \mathsf{ct}_1^{y})$.

Observe that the combined multiply-and-decrypt procedure uses ys and y. Now, consider what happens if we only have secret shares of them, which is exactly how the memory share is structured. Let $(\mathsf{ct}_0, \mathsf{ct}_1) = (g^{r'}, g^{-r's}(1+N)^x)$, then $\mathsf{ct}_0^{\langle ys \rangle_\sigma} \mathsf{ct}_1^{\langle y \rangle_\sigma} = g^{\langle 0 \rangle_\sigma}(1+N)^{\langle xy \rangle_\sigma}$. It might be tempting to think that discrete log with respect to $N+1$ can recover $\langle xy \rangle_\sigma \bmod N$. However, note that the definition of a subtractive share only guarantees that $\langle 0 \rangle_A - \langle 0 \rangle_B = 0$, and in general $\langle 0 \rangle_\sigma \neq 0$. Fortunately, using the distributed discrete log (DDLog) procedure [24, Algorithm 3.2], party σ can recover a value z_σ such that $z_A - z_B \equiv xy \bmod N$, so z_σ is a valid subtractive share of xy with overwhelming probability when $|xy| \ll N$. Specifically, if the RMS program is B-bounded for some $B \leq 2^{-\tau}N$, the failure probability of this procedure is upper bounded by $2^{-\tau}$.

Provided that $|xys| \ll N$ (this is where short exponent s is useful), we can use the same procedure to obtain $\langle xys \rangle_\sigma$ from the encryption of xs: $(g^r(1+N)^x, g^{-rs})$. At the end, this gives a procedure to obtain $\langle\!\langle xy \rangle\!\rangle_\sigma = (\langle xys \rangle_\sigma, \langle xy \rangle_\sigma)$ from $[\![x]\!]_\sigma$ and $\langle\!\langle y \rangle\!\rangle_\sigma$.

2.2.2. Share Synchronization to Achieve MKHSS. Recall that MKHSS allows two parties to generate independent public-secret key pairs, and each party creates input shares as encryptions under its own key. The problem comes when evaluating functions on such inputs, which could be supplied by either party and thus encrypted under different keys.

The approach taken by Couteau et al. [6] to achieve MKHSS is the following: During the evaluation procedure, first transform all encryptions in the input shares to be under some joint public key (this step is referred to as share *synchronization*), then use a procedure to bootstrap the first memory share of 1 (which contains a subtractive secret share of the joint secret key), and finally, evaluate the function.

Specifically, denote s_σ as the secret key of party σ, and the corresponding public key as $f_\sigma = g^{-s_\sigma}$. We define the joint secret key as $s = s_A s_B$. Observe that it is trivial for each party to then obtain the joint public key via a Diffie-Hellman key exchange, e.g., Alice computes $f = f_A^{s_B}$. In addition, it turns out that encryptions of xs_A under the public key f_A can be transformed into encryptions of xs under the *joint* public key f easily, via exponentiating the second component by s_B:

$$(\mathsf{ct}_{0,A}, \mathsf{ct}_{1,A}^{s_B}) = (g^r(1+N)^x, f_A^{-rs_B}) = (g^r(1+N)^x, f^{-r}).$$

Unfortunately, doing the same thing for encryptions of x results in an encryption of $-s_B x$:

$$(g^r, (f_A^{-r}(1+N)^x)^{-s_B}) = (g^r, f^{-r}(1+N)^{-s_B x}).$$

Couteau et al. [6] solved this by requiring both parties to sample the individual secret key to be of a special form: $s_\sigma = t_\sigma \cdot N + 1$. Observe that $s_\sigma \equiv 1 \bmod N$, and since the order of $N+1$ is N, $(1+N)^{-s_B x} \equiv (1+N)^x \bmod N^2$. It turns out that if we sample $t_\sigma \leftarrow\!\$\, [2^{2\lambda}]$, this remains secure under SEI. In the interest of space, we provide a formal proof that such secret key structure retains security in the full version [25].

A significant problem with this approach is that even when t_σ is short, the structure of s_σ means its size must be as large as the modulus N. To make matters worse, the joint secret key is a product $s = s_A s_B$, so it is even bigger. Recall that we require that after a multiplication, $xys \ll N$. This is impossible to satisfy regardless of the size of N.

Couteau et al. used a heavyweight solution to this problem: they encrypted xs using Damgård-Jurik-ElGamal (DJEG)—a generalization of Paillier-ElGamal with the ciphertext modulus being N^{w+1} instead of N^2. They specifically used the parameter $w = 3$, thereby increasing the ciphertext modulus from N^2 to N^4, adding significant computation overhead to many operations. Since the order of $1 + N$ is N^3 in $\mathbb{Z}^*_{N^4}$, we now only require $xys \ll N^3$. A further complication is that to ensure encryptions of x can still be synchronized, Paillier-ElGamal is used *just* for it. Even when the asymptotically efficient Karatsuba [26] multiplication algorithm is used, any group operation in N^4 is still at least $3\times$ slower than that in N^2.

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

The last step is to bootstrap the initial memory share $\langle\!\langle 1 \rangle\!\rangle_\sigma = (\langle s \rangle_\sigma, \langle 1 \rangle_\sigma)$, specifically $\langle s \rangle_\sigma$. This can be done using a procedure called Non-Interactive Multiplication (NIM) [6], [27].

2.3. Our Optimizations

We uncover several algorithmic insights to significantly improve the performance of MKHSS, including overcoming a major obstacle in Couteau et al., i.e., the need to use the modulus N^4. The largest modulus in our final construction is N^2, the same modulus used by Paillier's cryptosystem, and is optimal for constructions based on the DCR assumption (which uses the group $\mathbb{Z}^*_{N^2}$).

The reduced modulus size allows us to achieve a significant concrete improvement in both running time (due to faster group operations) and reduction in communication and memory usage (due to the smaller size of integers).

2.3.1. Deriving $\langle y \rangle_\sigma$ directly from $\langle ys \rangle_\sigma$. Consider the RMS multiplication procedure, which takes in $[\![x]\!]_\sigma$ and $\langle\!\langle y' \rangle\!\rangle_\sigma$ and produces a memory share of a product $y = xy'$: $\langle\!\langle y \rangle\!\rangle_\sigma = (\langle y \rangle_\sigma, \langle ys \rangle_\sigma)$. In Couteau et al., they derive $\langle y \rangle_\sigma$ and $\langle ys \rangle_\sigma$ separately. Computing *either* of the two requires two modular exponentiations, so the total cost to compute $\langle\!\langle y \rangle\!\rangle_\sigma$ is dominated by four exponentiations.

Instead, we derive $\langle y \rangle_\sigma$ directly from $\langle ys \rangle_\sigma$ *without* any exponentiations, reducing total cost of RMS multiplication from four down to two exponentiations. This appears challenging because for several reasons: (1) division by s is a nonlinear operation which is challenging to do in HSS, (2) s is also the secret key, which is not known to either party. Our insight is that because $|y| \ll N$ in all existing group-based HSS schemes, each party to locally reduce its share $\langle ys \rangle_\sigma$ modulo N and obtain $\langle y \rangle_\sigma$ with overwhelming probability.

More formally, our scheme ensures that $\langle ys \rangle$ is a random subtractive share using shared randomness between the parties (see Appendix A.2 and Appendix A.3). By definition, $\langle ys \rangle_A = \langle ys \rangle_B + ys$, so $\langle ys \rangle_A \bmod N = (\langle ys \rangle_B \bmod N + y \bmod N) \bmod N$. Since $\langle ys \rangle_B \bmod N$ is uniformly random, then *over the integers*, $\langle y \rangle_A - \langle y \rangle_B = \langle ys \rangle_A \bmod N - \langle ys \rangle_B \bmod N = y$, where the latter equality holds with probability at least $1 - 2^{-\tau}$ when $|y| \le 2^{-\tau} N$.

2.3.2. Removing Encryptions of x. Previously, an input share $[\![x]\!]_\sigma$ consisted of encryptions of xs and x. It turns out that the ability to derive $\langle xy \rangle_\sigma$ directly from $\langle xys \rangle_\sigma$ has an additional benefit: the input share can now be *just* an encryption of xs and not x.

Consider the RMS multiplication operation: we use $[\![x]\!]_\sigma$ and $\langle\!\langle y \rangle\!\rangle_\sigma$ to compute the memory share $\langle\!\langle xy \rangle\!\rangle_\sigma = (\langle xy \rangle_\sigma, \langle xys \rangle_\sigma)$. We describe the process to compute $\langle x'y \rangle_\sigma$, which we can substitute $x' \in \{x, xs\}$ to recover both $\langle xy \rangle_\sigma$ and $\langle xys \rangle_\sigma = \langle (xs)y \rangle_\sigma$:

1) Let $(\mathsf{ct}_0, \mathsf{ct}_1)$ be an encryption of x'.
2) Compute $\langle x'y \rangle_\sigma = \mathsf{DDLog}(\mathsf{ct}_0^{\langle ys \rangle_\sigma} \cdot \mathsf{ct}_1^{\langle y \rangle_\sigma})$.

Note that since we can derive $\langle xy \rangle_\sigma$ *directly* from $\langle xys \rangle_\sigma$ without invoking the procedure above, we only need to invoke the procedure on $x' = xs$ and *not* $x' = x$, removing the need for having additional encryptions of x.

2.3.3. Shortening the Secret Key. Recall that the reason Couteau et al. requires the secret key s_σ to be at least as large as N is because in the encryption of x, we require $(N+1)^s \equiv N + 1 \in \mathbb{Z}^*_{N^2}$, i.e., $s \equiv 1 \bmod N$. Their major restriction is overcome by our optimization in Section 2.3.2 that removes the need to use encryptions of x. However, it may seem that the constraint introduced in Section 2.3.1 that $y \le 2^{-\tau} N$ still prevents us from modifying the structure of s_σ. Nevertheless, since we are computing a B-bounded RMS program, we know $y \le B$, and B is often significantly smaller than N in practice (e.g., $B = 2$ and N being a 3072-bit integer). Let $B' = B \cdot 2^\tau$. Therefore, we can set the secret key to $s_\sigma = t_\sigma \cdot B' + 1$, and later set $\langle y \rangle_\sigma \leftarrow \langle ys \rangle_\sigma \bmod B'$ without breaking correctness. In the interest of space, we defer the reader to the full version [25] for a formal proof that this secret key structure does not harm security.

The effect of this optimization is doubled in the joint secret key $s = s_A s_B$. In addition, the length of s no longer depends on the length of N, which allows us to use Paillier-ElGamal ($w = 1$) instead of the generalized DJEG, and work solely over the group $\mathbb{Z}^*_{N^2}$ rather than $\mathbb{Z}^*_{N^4}$.

2.3.4. Reducing Exponent Length Further. The primary performance bottleneck of HSS is modular exponentiations, whose running time is linear in the size of the exponent. Every RMS multiplication in this optimized scheme requires computing $\mathsf{ct}_0^{\langle ys \rangle_\sigma} \bmod N^{w+1}$ and $\mathsf{ct}_1^{\langle y \rangle_\sigma} \bmod N^{w+1}$.

Section 2.3.3 automatically reduced $\langle y \rangle_\sigma$ to $B \cdot 2^\tau$ bits. However, $\langle ys \rangle_\sigma$ is still $\log_2(N)$ bits since it is just the output of DDLog. It turns out that a similar local modular reduction trick can be used to reduce the size of $\langle ys \rangle_\sigma$, speeding up the next RMS multiplication.

Specifically, $|xs| \le |xs_A s_B| \le B \cdot (B' \cdot 2^{2\lambda})^2 = B^3 \cdot 2^{4\lambda + 2\tau}$. To minimize correctness error, we need an extra 2^τ factor, so denote $B'' = B^3 \cdot 2^{4\lambda + 3\tau}$. We can then set $\langle ys \rangle_\sigma \leftarrow \langle ys \rangle_\sigma \bmod B''$ and retain correctness with overwhelming probability. This is similar to an optimization in [28, Section V.B] where they *transform* the output of DDLog to a smaller modulus. We remark that for practical parameter sizes used for ANIKE, $\log_2(B'') \approx \log_2(N)/3$, so this roughly provides a $3\times$ speedup.

2.3.5. Reducing Input Length to NIM. Recall that in order to obtain the initial memory share $\langle\!\langle 1 \rangle\!\rangle_\sigma$, each party needs to obtain a subtractive share of the joint secret key $s = s_A s_B$. Setting aside the details, a non-interactive multiplication (NIM) procedure allows Alice and Bob, holding inputs x, y respectively, to obtain a secret share of xy. Couteau et al. [6] obtains $\langle s \rangle_\sigma$ using the naïve way: by having Alice set $x = s_A$ and Bob set $y = s_B$.

Internally, NIM is implemented with expensive modular exponentiations, so it helps to shorten the NIM inputs x, y as much as possible. Recall our secret key is of a special structure $s_\sigma = t_\sigma \cdot B' + 1$, so it is natural to ask whether we

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

can set $x = t_A$ and $y = t_B$ as NIM inputs, obtain $\langle t_A t_B \rangle_\sigma$, and do some manipulations to transform it into $\langle s \rangle_\sigma$.

To achieve this, we observe that $s = s_A s_B = (t_A B' + 1)(t_B B' + 1) = t_A t_B (B')^2 + (t_A + t_B)B' + 1$, and that $\langle t_A + t_B \rangle_A = t_A$, $\langle t_A + t_B \rangle_B = -t_B$ is a valid subtractive share. Finally, the result $\langle s \rangle_\sigma$ can be obtained as a linear combination: $\langle s \rangle_\sigma = \langle t_A t_B \rangle_\sigma (B')^2 + \langle t_A + t_B \rangle_\sigma B' + \langle 1 \rangle_\sigma$.

2.4. Correctness

We now pay close attention to the condition which ensures the correctness of our optimized scheme. Let l_N denote the bit-length of the modulus N, and w be the DJEG parameter, such that the ciphertext modulus is N^{w+1}. In order to ensure the $\langle ys \rangle_\sigma$ parts of a memory share of y remain subtractive shares over $\mathbb{Z}$ after each DDLog procedure, we need $|ys| \leq N^w$ even when we *do not* artificially reduce modulo B'' in Section 2.3.4. Therefore, we need $N^w \geq B'' = B^3 \cdot 2^{4\lambda+3\tau}$. In other words, $wl_N \geq 3\log_2(B) + 4\lambda + 3\tau$.

To deal with arbitrarily large message bounds B, we may increase either l_N or w, whichever is more efficient. We discuss in Section 4.1.3 how we optimally increase those parameters to tolerate large bounds B for evaluation.

We refer readers who are interested in seeing the full definition of MKHSS, details regarding our construction, and our proof of correctness and security to Appendix C.

3. Technical Overview: ANIKE

Attribute-based non-interactive key exchange (ANIKE) is a primitive that solves the following problem: two parties, each holding private inputs x_A and x_B, want to agree on a common key if and only if their inputs satisfy some policy P: $P(x_A, x_B) = 0$, without revealing to each other any information beyond whether P is satisfied. This primitive naturally lends itself to applications to location-based and fuzzy password-authenticated key exchange (PAKE), by observing that P can be a predicate that determines whether x_A and x_B are *close* in some metric.

Moreover, ANIKE is *non-interactive* in the same sense as a non-interactive key exchange (NIKE) [8] (e.g., Diffie-Hellman [9]). The key exchange protocol works by having each party run an algorithm AttrKeyDer to create a public encoding pe_σ of its private attribute x_σ, exchange the public encodings, and the parties locally deriving a key via AttrKeyGen. The protocol is illustrated in Figure 1.

Couteau et al. [6] showed the theoretical feasibility of ANIKE from MKHSS that supports the same class of predicates as MKHSS, i.e., RMS programs, or logarithmic-space computation.

3.1. Our Contributions

We show that ANIKE can be concretely realized with similar efficiency as our MKHSS scheme on the same predicate. In addition, we show that our ANIKE can realize predicates with real-world applications to a geolocation-based key exchange and fuzzy PAKE.

3.1.1. Application: Geolocation-Based Key Exchange. Consider the case where each party's input is a latitude-longitude pair $x_\sigma = (u_\sigma, v_\sigma)$, and the predicate $P(x_A, x_B)$ is satisfied when x_A and x_B represent geographically close locations. A canonical application is to use location services (e.g., GPS) to authenticate a key exchange between geographically close devices, where the devices need not be physically close, e.g., on different floors in a skyscraper. We also present another application that uses self-reported location information instead of location services. Consider a traveler who wishes to contact local residents near his travel destination to discuss arrangements for a homestay (à la AirBnB). The traveler may wish to only consider stays within a certain distance d from his destination (e.g., a conference venue), so we only want parties that satisfy this location proximity to successfully complete the key exchange. The resident wants to hide her home address (until she agrees to host a traveler) to ensure privacy, and the traveler may similarly not wish to disclose the exact destination. The non-interactivity of ANIKE allows each party to engage in this protocol asynchronously without being simultaneously online.

3.1.2. Application: Fuzzy PAKE. Password-authenticated key exchange (PAKE) [29] allows two parties to agree on a common key using a shared password. It is useful when a user wants to configure two personal devices to agree on the same key. Traditional password-based protocols, such as that of Facebook [30], often allow authentication under a small set of common typos to enhance usability. However, it is challenging to equip PAKE with a fuzzy notion of equality. Unlike plain PAKE, prior constructions of fuzzy PAKE [10] for rich notions of fuzziness are based on MPC and require several rounds of interaction. See Section 1.1 for the importance of non-interactivity.

We focus on the case where the password is a passphrase. It is well known that passphrases provide a good balance between memorability and security.[3] We model a passphrase as a list of relatively short strings, whose contents can be arbitrary characters from an exponentially large set (e.g., lowercase letters, ASCII, UTF-16, etc.). In this model, we are able to successfully complete a key exchange despite a limited number of substitutions, transpositions, insertions, and deletions, and do so for realistic passphrase and word lengths.

Importance of fuzziness. A study [31] found that the lack of fuzzy matching significantly increase login delay (and prevented some users from *ever* completing a successful login) *even* in the setting where a user is given nearly instant feedback when an incorrect password is entered. Note that the non-interactive nature of the fuzzy PAKE protocol *delays* such feedback in most scenarios. For example, in the scenario of key agreement between personal devices, the user only knows if the key exchange succeeds *after* both personal devices use the key in another protocol.

3. Captured by this XKCD comic: https://xkcd.com/936/.

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

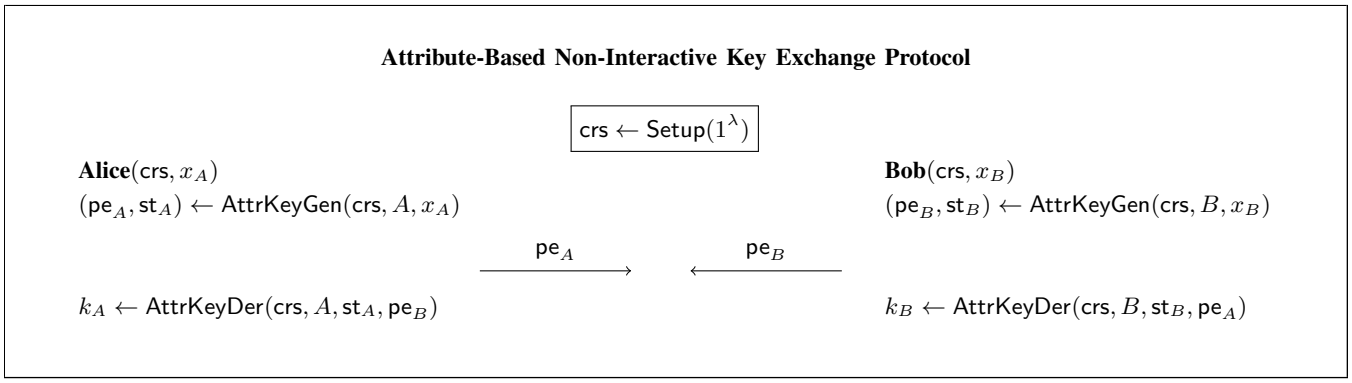


Figure 1: The ANIKE protocol. See Definition 2 for the formal syntax of the algorithms AttrKeyGen and AttrKeyDer.

3.1.3. Multi-Factor Authenticated Key Exchange. We show it is possible to combine arbitrary predicates via logical conjunctions and disjunctions. This enables a form of *multi-factor* authenticated key exchange, e.g., keys agree if the location is close *and* the password match. For example, a system administrator may wish to configure two corporate-owned devices to agree on the same key using a remembered password, where the devices are located in different rooms in the same building. In this scenario, using a combined location proximity and password will provide better security.

There are other interesting compositions. For the home-stay example, the traveler may have additional requirements such as an acceptable price range, and such a numerical interval comparison can be seen as a simpler 1-dimensional case of the location proximity predicate.

We stress that since this composition works at the *predicate* level, it requires no extra rounds of communication or protocol modifications: the non-interactivity and reusability property of ANIKE is preserved. The cost (public encoding size and computation time) is at most a sum over the cost of running the protocol over individual predicates.

3.2. Starting Point: Couteau et al.

Couteau et al. [6] provides a theoretical solution to ANIKE via black-box use of an MKHSS scheme with external security, which our MKHSS scheme achieves. External security requires that the subtractive shares of the output look pseudorandom conditioned on the value of the output. We describe their idea below:

A natural idea to transform MKHSS into key exchange is to observe that subtractive shares $\langle x \rangle = (\langle x \rangle_A, \langle x \rangle_B)$ have the following nice property: $\langle x \rangle_A = \langle x \rangle_B$ if and only if $x = 0$. Since the output of MKHSS is a subtractive share, an extremely naïve solution is to make MKHSS compute the policy function $P(x, y)$ directly and let $k_\sigma = \langle P(x, y) \rangle_\sigma$.

One problem is that even when $P(x, y) \neq 0$, the shares are still correlated. For example, if Alice knows that $P(x, y)$ is a small integer, she can easily brute force and obtain Bob's key. An immediate fix is to have each party encode a secret shift value $\Delta_\sigma \in [2^\lambda]$ into an additional input share, and have MKHSS compute $P(x, y) \cdot \Delta_A \cdot \Delta_B$ instead.

Recall that reusability is a crucial requirement of ANIKE. Unfortunately, the encoding of the secret shift is not reusable. Consider a client-server authentication protocol that uses ANIKE as a building block, where after two parties obtain keys k_A, k_B, one party (say Alice, the client) sends over her key k_A to the server Bob, who then checks whether $k_A = k_B$. If Bob is the adversary, he can calculate $k_B - k_A = \Delta_A \cdot \Delta_B \cdot P(x, y)$. Suppose Alice failed the authentication procedure, and $P(x, y) = 1$, then Bob could recover Alice's long-term secret shift $\Delta_A = (k_B - k_A)/\Delta_B$. This compromises the security of future key exchanges.

To achieve reusability, instead of encoding Δ_σ directly in an input share, Couteau et al. makes each party encode a PRF key K_σ in its long-term public encoding. Then, for each $\sigma \in \{A, B\}$, compute $\Delta_\sigma = F(K_\sigma, \mathsf{id}_A || \mathsf{id}_B || P)$ inside the RMS program. This way, for every key exchange session, the PRF generates an independently random shift.

3.3. Making Generic ANIKE Efficient

The major inefficiency in their construction is the need to evaluate a pseudorandom function (PRF) *homomorphically* over values inside the RMS program, making it impractical to implement, *even* with an efficient MKHSS scheme.

Instead, we observe that only a small modification is needed to secure the otherwise insecure approach where each party σ encodes Δ_σ. The idea is to let the key be a *hash* of the subtractive share along with a unique value for each session: $k_\sigma = H(\langle P(x, y) \cdot \Delta_A \cdot \Delta_B \rangle_\sigma \| \mathsf{id}_A \| \mathsf{id}_B \| P)$. We model the hash function as a random oracle, and security follows from the correlation-robustness of hashing.

However, when we instantiate our MKHSS scheme for this predicate, we require increasing the bound to $B \geq 2\lambda$ so that correctness (and security) hold(s). Fortunately, due to the fact that our memory share of $P(x, y)$ is already of the format $\langle\!\langle P(x, y) \rangle\!\rangle_\sigma = (\langle P(x, y) \cdot s \rangle_\sigma, \langle P(x, y) \rangle_\sigma)$, we can hash the former element, and use s in place of $\Delta_\sigma \cdot \Delta_{1-\sigma}$. It provides the same security guarantee, since the joint secret $s = s_A s_B$ is unknown to any single party, again relying on modeling the hash as a random oracle.

Lastly, we replace $\mathsf{id}_A \| \mathsf{id}_B$ with a key k_{nike} obtained through any non-interactive key exchange (e.g., Diffie-

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

Hellman) that runs in parallel to our protocol. This serves not only as a replacement of the identity strings, but crucially ensures that the resulting keys look pseudorandom to external parties even when the predicate is satisfied, without extra requirements on the output of the MKHSS scheme. In the end, the key derivation procedure computes $k_\sigma = H(\langle P(x,y)\cdot s\rangle_\sigma \| k_{\mathsf{nike}} \| P)$.

3.4. Geolocation-Based Key Exchange

After showing a generic ANIKE construction, it remains to show how to use it with practically interesting policies from efficiently computable RMS programs. The first predicate we consider is location proximity. We model a party's location $\mathbf{x}_{\boldsymbol{\sigma}}$ as a D-dimensional vector of L-bit integers, and the predicate $P(\mathbf{x_A}, \mathbf{x_B}) = 0$ if and only if $\mathbf{x_A}$ and $\mathbf{x_B}$ are within a distance d according to some metric.

For simplicity of presentation, we focus on a simple example where the location of each party can be described as a two-dimensional coordinate in an Euclidean plane, i.e., each party's attribute is a vector of two integers: $\mathbf{x}_{\boldsymbol{\sigma}} = (u_\sigma, v_\sigma)$. We define closeness via the Chebyshev distance, i.e., $\mathsf{CD}(\mathbf{x_A}, \mathbf{x_B}) = \max\{|u_A - u_B|, |v_A - v_B|\}$. In other words, two parties agree on a key if $\mathbf{x_B}$ is located within a square area centered around $\mathbf{x_A}$. Observe that $\mathsf{CD}(\mathbf{x_A}, \mathbf{x_B}) < d \iff |u_A - u_B| < d \wedge |v_A - v_B| < d$. In fact, we can decompose a single comparison further: $|u_A - u_B| < d \iff u_A > u_B - d \wedge u_A < u_B + d$.

By letting each party $\sigma \in \{A, B\}$ provide shares that encode integers $u_\sigma, u_\sigma - d, u_\sigma + d, v_\sigma, v_\sigma - d, v_\sigma + d$, this reduces to performing comparison $x < y$ using an RMS program. In fact, we further reduce the number of shares by letting the parties (denote Alice and Bob) play asymmetric roles. Alice just needs to share u_A and v_A, and Bob just needs to share $u_\sigma \pm d$ and $v_\sigma \pm d$.

Lastly, we show how to do the integer comparison $x < y$. We let the input shares encode a binary representation of x and y, so integer comparison can be seen as a branching program of size linear in the bit lengths of x and y using the simple algorithm of bit-by-bit comparison, and we provide an explicit RMS program in Construction 1.

To see correctness, observe that in line 7, LessThan recursively defines $z_i = (\overline{x}_i\,\overline{y}_i + x_i y_i) z_{i-1} = z_{i-1} \cdot \mathbb{1}(x_i = y_i)$. Thus, $z_i = z_0 \cdot \mathbb{1}(x_1 \dots x_i = y_1 \dots y_i)$, i.e., if $z_0 = 1$ and the prefix of length i matches.

Observe that $x < y$ if and only if the binary decompositions differ, and in the first bit position i where they differ, $x_i = 0$ and $y_i = 1$. Using logic notation, this can be described as $\mathbb{1}(x < y) = \bigvee_{i=1}^{L} (\overline{x}_i \wedge y_i \wedge \mathbb{1}(x_1 \dots x_{i-1} = y_1 \dots y_{i-1}))$, where it is easy to see that the clauses across different indices i are mutually exclusive. Therefore, line 8 can be seen as computing this big disjunction via a summation.

If the goal is to just compute $x < y$, we can set $\langle\!\langle z_0 \rangle\!\rangle_\sigma = \langle\!\langle 1 \rangle\!\rangle_\sigma$, but we made the algorithm take in $\langle\!\langle z_0 \rangle\!\rangle_\sigma$ as an argument to allow computing logical conjunctions over multiple comparisons (recall it is not possible to multiply memory values in an RMS program). For example, to compute $x < y \ \wedge \ v < w$, we can do:

1) $\langle\!\langle \mathbb{1}(x < y) \rangle\!\rangle_\sigma \leftarrow \mathsf{LessThan}(\{\!\{x\}\!\}, \{\!\{y\}\!\}, \langle\!\langle 1 \rangle\!\rangle_\sigma)$
2) **return** $\mathsf{LessThan}(\{\!\{v\}\!\}, \{\!\{w\}\!\}, \langle\!\langle \mathbb{1}(x < y) \rangle\!\rangle_\sigma)$

which will be needed for our location proximity ANIKE predicate. This will also enable *composing* the location predicate with other extra requirements. Using the homestay example, it could be price ranges, amenities, etc. In addition, using more conjunctions allows this predicate to be generalized to higher dimensions $D \geq 3$, by having Alice provide shares $x_A^{(1)}, \dots, x_A^{(D)}$ (instead of u_A, v_A) and Bob provide shares $x_B^{(1)} \pm d, \dots, x_B^{(D)} \pm d$ (instead of $u_B \pm d, v_B \pm d$).

Using a similar technique, multiple location predicates can also be composed using logical disjunctions, e.g., allowing key exchange between devices located in any one of the corporate headquarters.

3.4.1. Generalizations. We presented a predicate that checks whether the Chebyshev distance is under a threshold d in Euclidean space, i.e., whether $\mathbf{x_B}$ is within a hypercube region around $\mathbf{x_A}$. We define this square region as the *accepting region* of this predicate. We remark that our scheme is very versatile and supports many extensions that alter this *accepting region* to support practical scenarios, like geographical coordinates.

Hyper-rectangular accepting regions. We can easily make this accepting region an arbitrary hyperrectangle by having different thresholds $d_1, \dots, d_D$ for dimensions 1 through D.

Then, observe that for a key exchange to succeed, we only need the shares $x_A^{(1)}, \dots, x_A^{(D)}$ from one party (say Alice), and $x_B^{(1)} \pm d_1, \dots, x_B^{(D)} \pm d_D$ from Bob. Therefore, instead of setting $d_1, \dots, d_D$ as system parameters, we can allow one party (here Bob) to pick them. This is already useful in the homestay example, where the traveler may

Construction 1: RMS program to compute whether $a < b$.

The L-bit integers x, y are encoded in synchronized input shares that encode their binary decompositions: $\{\!\{\mathbf{x}\}\!\} = (\{\!\{x_1\}\!\}, \dots, \{\!\{x_L\}\!\})$ and $\{\!\{\mathbf{y}\}\!\} = (\{\!\{y_1\}\!\}, \dots, \{\!\{y_L\}\!\})$. $\langle\!\langle a \rangle\!\rangle_\sigma$ denotes a memory share of a. We denote $\overline{a} := 1 - a$.

$\mathsf{LessThan}(\{\!\{\mathbf{x}\}\!\}, \{\!\{\mathbf{y}\}\!\}, \langle\!\langle z_0 \rangle\!\rangle_\sigma)$

1 : **for** i **in** $\{1, \dots, L\}$,
2 : $\quad \langle\!\langle x_i z_{i-1} \rangle\!\rangle_\sigma \leftarrow \{\!\{x_i\}\!\} \cdot \langle\!\langle z_{i-1} \rangle\!\rangle_\sigma$
3 : $\quad \langle\!\langle \overline{x}_i z_{i-1} \rangle\!\rangle_\sigma \leftarrow \langle\!\langle z_{i-1} \rangle\!\rangle_\sigma - \langle\!\langle x_i z_{i-1} \rangle\!\rangle_\sigma$
4 : $\quad \langle\!\langle \overline{x}_i y_i z_{i-1} \rangle\!\rangle_\sigma \leftarrow \{\!\{y_i\}\!\} \cdot \langle\!\langle \overline{x}_i z_{i-1} \rangle\!\rangle_\sigma$
5 : $\quad \langle\!\langle \overline{x}_i\,\overline{y}_i z_{i-1} \rangle\!\rangle_\sigma \leftarrow \langle\!\langle \overline{x}_i z_{i-1} \rangle\!\rangle_\sigma - \langle\!\langle \overline{x}_i\, y_i z_{i-1} \rangle\!\rangle_\sigma$
6 : $\quad \langle\!\langle x_i y_i z_{i-1} \rangle\!\rangle_\sigma \leftarrow \{\!\{y_i\}\!\} \cdot \langle\!\langle x_i z_{i-1} \rangle\!\rangle_\sigma$
7 : $\quad \langle\!\langle z_i \rangle\!\rangle_\sigma \leftarrow \langle\!\langle \overline{x}_i\,\overline{y}_i z_{i-1} \rangle\!\rangle_\sigma + \langle\!\langle x_i y_i z_{i-1} \rangle\!\rangle_\sigma$
8 : $\langle\!\langle z_0 \cdot \mathbb{1}(x < y) \rangle\!\rangle_\sigma \leftarrow \sum_{i=1}^{L} \langle\!\langle \overline{x}_i y_i z_{i-1} \rangle\!\rangle_\sigma$
9 : **return** $\langle\!\langle z_0 \cdot \mathbb{1}(x < y) \rangle\!\rangle_\sigma$

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

want to decide the distance depending on commute time preferences.

Geographical coordinates. Geolocation coordinates are spherical rather than Euclidean coordinates. Therefore, to compare whether two latitude-longitude pairs are close (under some relatively small metric distance threshold m), we need to account for the conversion factor α such that a metric difference of 1 meter corresponds to an angular difference of α degrees in latitude and β degrees in longitude, i.e., set $d_1 = \alpha m$ and $d_2 = \beta m$. While α is fixed, β depends on the approximate latitude of both parties, so when m is small (implying both parties must lie on similar latitude for authentication to succeed), we need one party to provide d_2, which our scheme supports.

An alternative approach is to convert the spherical coordinates to a *three-dimensional* Euclidean coordinate describing the displacement from Earth's center, if one is willing to tolerate the additional cost of approximating Euclidean distance with Chebyshev distance in three dimensions. The maximum deviation between Chebyshev distance and Euclidean distance is $\sqrt{D}$, which in $D = 3$ dimensions, is 73%.

Approximate Euclidean distance. Observe, we can make the accepting region an intersection of many rotated hypercubes, by having each party provide shares of their coordinates in rotated coordinate systems, and making the predicate a logical conjunction. It is easy to see that in two dimensions, the intersection of sufficiently many rotated squares approximate a circle, so this can be used to arbitrarily approximate a Euclidean distance threshold in low dimensions, providing a more precise matching.

3.5. Fuzzy PAKE

3.5.1. Formalization. We define a passphrase as a sequence of W words of length at most L, whose characters come from an alphabet Σ. Formally, a passphrase is a vector of words $\mathbf{x} = x_1, \ldots, x_W \in (\Sigma \sqcup \{\perp\})^L$, where each word x_i is a string potentially padded with $\perp$ symbols until its length is L.

We describe the predicate $P(\mathbf{x}, \mathbf{y})$ we use to capture real-world mistakes, which is parameterized by two thresholds Q and T:

1) For each index i, we permit x_i and y_i to differ by at most Q single character substitutions without penalty. This accounts for substitution and (to a lesser extent) transposition errors.
2) Let E be the number of corresponding words that differ by more than Q substitutions. We define $P(\mathbf{x}, \mathbf{y}) = 1$ if and only if $E \leq T$. This way, we can tolerate up to T insertions, deletions, and even forgetting entire words.

3.5.2. Warmup: Hamming Distance. We first show how to construct an RMS program for a simple predicate where $P(\mathbf{x}, \mathbf{y}) = 1$ if the Hamming distance between two binary strings (treated as a vector of bits) $\mathbf{x}, \mathbf{y} \in \{0,1\}^L$ is strictly greater than T, and $P(\mathbf{x}, \mathbf{y}) = 0$ otherwise.

Construction 2: RMS program to compute whether the Hamming distance between two strings $x, y \in \{0,1\}^L$ is stricly greater than a threshold T.

The strings $x, y \in \{0,1\}^L$ are represented by vectors of input shares $\{\!\{\mathbf{x}\}\!\} = (\{\!\{x_1\}\!\}, \ldots, \{\!\{x_L\}\!\})$ and $\{\!\{\mathbf{y}\}\!\} = (\{\!\{y_1\}\!\}, \ldots, \{\!\{y_L\}\!\})$. $\langle\!\langle v \rangle\!\rangle_\sigma$ denotes a memory share of v.

$\mathsf{ExceedsThreshold}_L(\{\!\{\mathbf{x}\}\!\}, \{\!\{\mathbf{y}\}\!\}, T, \langle\!\langle z_0 \rangle\!\rangle_\sigma)$

1 : **for** j **in** $\{0, \ldots, T+1\}$,
2 : $\quad \langle\!\langle a_{0,j} \rangle\!\rangle_\sigma \leftarrow \langle\!\langle z_0 \rangle\!\rangle_\sigma$ **if** $j = 0$ **else** $\langle\!\langle 0 \rangle\!\rangle_\sigma$
3 : **for** i **in** $\{1, \ldots, L\}$,
4 : $\quad \langle\!\langle z a_{i-1,(-1)} \rangle\!\rangle_\sigma \coloneqq \langle\!\langle 0 \rangle\!\rangle_\sigma$
5 : $\quad$ **for** j **in** $\{0, \ldots, T\}$,
6 : $\quad\quad \langle\!\langle z a_{i-1,j} \rangle\!\rangle_\sigma \leftarrow \mathsf{Differ}(\{\!\{x_i\}\!\}, \{\!\{y_i\}\!\}, \langle\!\langle a_{i-1,j} \rangle\!\rangle_\sigma)$
7 : $\quad\quad \langle\!\langle \overline{z} a_{i-1,j} \rangle\!\rangle_\sigma \leftarrow \langle\!\langle a_{i-1,j} \rangle\!\rangle_\sigma - \langle\!\langle z a_{i-1,j} \rangle\!\rangle_\sigma$
8 : $\quad\quad \langle\!\langle a_{i,j} \rangle\!\rangle_\sigma \leftarrow \langle\!\langle z a_{i-1,j-1} \rangle\!\rangle_\sigma + \langle\!\langle \overline{z} a_{i-1,j} \rangle\!\rangle_\sigma$
9 : $\langle\!\langle z_0 P(x,y) \rangle\!\rangle_\sigma \leftarrow \sum_{i=0}^{L-1} \langle\!\langle z a_{i,T} \rangle\!\rangle_\sigma$
10 : **return** $\langle\!\langle z_0 P(x,y) \rangle\!\rangle_\sigma$

$\mathsf{Differ}(\{\!\{x\}\!\}, \{\!\{y\}\!\}, \langle\!\langle z_0 \rangle\!\rangle_\sigma)$

1 : $\{\!\{x - y\}\!\} \leftarrow \{\!\{x\}\!\} - \{\!\{y\}\!\}$
2 : $\langle\!\langle (x-y) z_0 \rangle\!\rangle_\sigma \leftarrow \{\!\{x - y\}\!\} \cdot \langle\!\langle z_0 \rangle\!\rangle_\sigma$
3 : $\langle\!\langle (x-y)^2 z_0 \rangle\!\rangle_\sigma \leftarrow \{\!\{x - y\}\!\} \cdot \langle\!\langle (x-y) z_0 \rangle\!\rangle_\sigma$
4 : **return** $\langle\!\langle (x-y)^2 z_0 \rangle\!\rangle_\sigma$

First, we observe that the natural way to compute this using pseudocode is to declare a zero-initialized accumulator that stores the number of differing bits, iterate through the string, and in each iteration, increment the accumulator conditioned on the bits differing.

The idea is to store the accumulator as a one-hot encoding in L memory values. Then in each iteration, update the accumulator based on the input bit values. We present a detailed algorithm in Construction 2. To do single-level threshold matching, we invoke $\mathsf{ExceedsThreshold}_L(\{\!\{\mathbf{x}\}\!\}, \{\!\{\mathbf{y}\}\!\}, T, \langle\!\langle 1 \rangle\!\rangle_\sigma)$.

To verify correctness, observe the invariant that $\mathbf{a_i} = (a_{i,0}, \ldots, a_{i,T})$ is a one-hot encoding of the number of differing bits in the first i bits of x and y if the value is at most T, and zero otherwise. In addition, $z a_{i,T}$ can be viewed an *overflow flag* for the accumulator $\mathbf{a_i}$ that indicates if an overflow occurred exactly after reaching the i-th bit, so summing over this value for all i determines whether the threshold T was ever exceeded.

3.5.3. Supporting Multiple Thresholds. To support more than one level of threshold, we observe that we can replace Differ with recursive calls to ExceedsThreshold inside itself.

We focus on a concrete example that is almost our two-level passphrase threshold (i,e. L words each of W characters), except the alphabet is still binary: $\Sigma \in \{0,1\}$: where we allow Q incorrect characters (bits in this simplified

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

example) and T incorrect words.

We first define $\mathsf{ExceedsThreshold}_W$ to be essentially the definition in Construction 2, except with W in place of L. We then redefine $\mathsf{ExceedsThreshold}_L$ to instead take in an $L \times W$ *matrix* of input values $\{\{\mathbf{X}\}\}$ and $\{\{\mathbf{Y}\}\}$, where $X_{i,j}$ is the j-th character of the i-th word. We then replace line 6 of $\mathsf{ExceedsThreshold}_L$ with a recursive call to $\mathsf{ExceedsThreshold}_W$ as follows

$$\langle\langle z \rangle\rangle_\sigma \leftarrow \mathsf{ExceedsThreshold}_W(\{\{\mathbf{x_i}\}\}, \{\{\mathbf{y_i}\}\}, \langle\langle a_{i-1,j} \rangle\rangle_\sigma),$$

where $\mathbf{x_i}$ and $\mathbf{y_i}$ are the i-th rows of the matrices $\mathbf{X}$ and $\mathbf{Y}$, respectively.

Lastly, we observe that to extend to three or more levels, we can make $\mathbf{X}$ and $\mathbf{Y}$ tensors, and recurse for more levels.

3.5.4. Supporting Arbitrary Alphabets. To extend the above program to an exponentially-large alphabet Σ, we first encode each character as a binary string of b bits. It suffices to have $b = \log_2(|\Sigma| + 1)$, i.e., every character in Σ and the special $\perp$ symbol can be encoded. We then treat character comparison as itself a trivial threshold matching problem with the threshold being zero (a single bit of difference makes two characters unequal). This means that our fuzzy PAKE example can be thought of as having three levels of threshold (0, Q, and T), and the input is a three-dimensional tensor of size $L \times W \times b$.

3.5.5. Optimizations. If we put together the accumulator values $a_{i,j}$ as a matrix $\mathbf{A}$ (i.e., stacking together one-hot encodings $\mathbf{a_i}$ as column vectors), then since the accumulator value starts at zero and increases by at most one in every iteration, it is easy to see that the lower left triangle of the matrix always consists of all zeroes. In addition, the upper right triangle of the matrix does not contribute to the output. Therefore, Line 6 in Construction 2 need not be executed for every combination of i, j. This saves calls to Differ, in addition to *recursive* calls to ExceedsThreshold, which provides significant savings in the number of RMS multiplications.

4. Evaluation

We implemented in C++ our MKHSS scheme *in addition* to a naïve implementation of the Couteau et al. [6] construction from DCR, which we denote as the baseline.[4] We provide empirical and analytical performance comparisons, using the experimental setup described in Section 4.1. We denote $\lg(\cdot)$ as the base-2 logarithm.

4.1. Experimental Setup

4.1.1. Implementation Details. We use binomial coefficients to compute $(N+1)^x \bmod N^{w+1}$. This optimization is well-known in prior work [32], so we adopt this optimization for both our implementation and the baseline.

4. https://github.com/kevin-he-01/mkhss/

Whenever modular exponentiation is repeated with the same base two or more times, we precompute powers of this base to speed up future operations. Specifically, we use either the fixed-base windowing or the fixed-base comb method [33], whichever is faster. Such a *fixed-base exponentiation* optimization is well-known [4], so we apply it to both our scheme and the baseline.

For MKHSS, we compute the joint public key $f = g^{-s_A s_B}$ once when initializing the RMS program, and reuse the same value when synchronizing all sender input shares. We apply this optimization to both our scheme and the baseline to ensure comparability between the schemes.

For ANIKE, we instantiate the internal NIKE scheme (Section 3.3) from an elliptic-curve Diffie-Hellman (ECDH) key exchange using the NIST P-256 curve.

4.1.2. Hardware. We use the following two machines:

1) *Laptop.* This is an x86_64 Linux machine running Ubuntu 22.04 on an AMD Ryzen 7 7735U processor, equipped with 32 GiB of memory. It runs the experiments to compare our scheme against the baseline.
2) *AWS.* This is an AWS bare metal instance of type `c7i.metal-24xl`, an x86_64 Linux machine running Amazon Linux 2023 with support for the AVX512-IFMA instruction set. It runs the experiments to demonstrate speedup using an efficient vectorized modular multiplication algorithm [34].

We only use a single core for both machines.

4.1.3. Parameters. We pick the computational and statistical security parameters to be $\lambda = 128$, $\tau = 128$ respectively.

The parameter B determines the largest integer (by absolute value) allowed during the computation for an RMS program. We vary $\lg(B) \in \{1, 512, 1024, 2048\}$ to show that our scheme scales to RMS programs that use integers ranging from 1 to 2048 bits long. We remark that all of our applications to key exchange only need $\lg(B) = 1$ because they compute on values in binary form.

For all values of B, we set l_N, the bit length of the modulus N, to be at least 3072 bits (possibly larger), which provides at least 128 bits of security against factoring attacks. In addition, recall from Section 2.4 that to ensure correctness, we need to ensure $wl_N \geq 3\log_2(B) + 4\lambda + 3\tau$, where $w \geq 1$ is an integer parameter that we can freely pick to optimize the scheme with no impact on security as long as the correctness condition holds. For large values of $\lg(B)$, we either increase l_N or the DJEG parameter w (See Appendix A.1) in our scheme:

1) For $\lg(B) = 1024$, we set $w = 1$ and $l_N = 3968$.
2) For $\lg(B) = 2048$, we set $w = 2$ and $l_N = 3520$.
3) For all other values $\lg(B) \leq 512$, we set $w = 1$ and $l_N = 3072$ for our scheme.

We set $l_N = 3072$ and $w = 3$ for the baseline on all parameters, since correctness of the baseline will not break down for our choices of $\lg(B)$.

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

TABLE 1: Correspondence between descriptive procedure names and names in our full construction in Construction 4.

Descriptive Name	Name in Construction 4
Generate CRS	$\mathsf{MKHSS.Setup}$
Generate key pair	$\mathsf{MKHSS.KeyGen}$
Encode an input share	$\mathsf{MKHSS.Share}$
Init RMS program (Alice)	$\mathsf{InitRMS}\ (\sigma = A)$
Init RMS program (Bob)	$\mathsf{InitRMS}\ (\sigma = B)$
Sync sender input share	$\mathsf{ExpLinEncS}$
Sync received input share	$\mathsf{ExpLinEncR}$
RMS Program Operation	
Add two inputs	$\mathsf{DEval.ScaleAdd_I}(a = 1, b = 1)$
Subtract two inputs	$\mathsf{DEval.ScaleAdd_I}(a = 1, b = -1)$
Add two memory values	$\mathsf{DEval.ScaleAdd_M}(a = 1, b = 1)$
Multiply memory and input	$\mathsf{DEval.Mult}$
Convert input to memory	$\mathsf{DEval.Convert}$

TABLE 2: The running time of procedures for our MKHSS scheme versus the baseline for $\lg(B) = 1$ on our *laptop*.

Procedure	Ours	Baseline	Our speedup
Generate CRS $*$	28.5 ms	251.9 ms	8.8×
Generate key pair	7.0 ms	104.5 ms	15×
Encode an input share	2.6 ms	10.3 ms	4.0×
Init RMS program (Alice)	4.4 ms	96.4 ms	22×
Init RMS program (Bob)	3.1 ms	50.3 ms	16×
Sync sender input share	1.3 ms	5.2 ms	4.0×
Sync received input share	1.8 ms	61.8 ms	35×
RMS Program Operation	Ours	Baseline	Our speedup
Add two inputs	9.5 μs	38.3 μs	4.0×
Subtract two inputs	67.3 μs	341.6 μs	5.1×
Add two memory values	19.6 ns	63.8 ns	3.3×
Multiply memory and input	5.0 ms	224.6 ms	45×
Convert input to memory	4.2 ms	167.8 ms	40×

*: Excluding the time to generate the two safe primes p and q.

4.2. Multi-Key HSS

Throughout the evaluation, we refer to MKHSS procedures using descriptive names. In Table 1, we show the correspondence between descriptive names and formal names from the full construction (Construction 4).

For $\lg(B) = 1$, we compare the running time to complete each procedure in the MKHSS construction and each RMS program instruction in Table 2 using our *laptop* machine.

The numbers are reported for running each procedure without precomputed values for fixed-base exponentiation, modeling application scenarios where each procedure is only invoked once. See Section 4.4 for an application that takes significant advantage of fixed-base exponentiation.

Since the performance of our multi-key HSS scheme depends on the bound B, we vary $\lg(B)$ from 1 to 2048 to show the scalability of our scheme when computing on very large integers in Figure 2 and show consistent improvement over the baseline. The baseline supports all these parameter sets with the same running time.

Lastly, we compare the size of keys, input, and memory shares for varying $\lg(B)$ in Figure 3.

4.2.1. Analytical Performance Comparison. To highlight our advantage over the baseline, we only consider parameter ranges where the correctness condition (Section 2.4) holds for $w = 1$. For our concrete parameter choices in Section 4.1.3, this means $\lg(B) \leq 725$.

We remark that a similar analysis can be done to show an advantage over the baseline for any $\lg(B)$ value where we only need a smaller w compared to the baseline (e.g., $w = 2$ vs $w = 3$ for $\lg(B) = 1024$). In addition, it is reasonable to assume the integers are under 725 bits in practice. In fact, our applications to key exchange only require $\lg(B) = 1$.

Running Time. The most expensive procedures in both our scheme and the baseline are group operations, and especially modular exponentiations. Since the baseline works in the group $\mathbb{Z}^*_{N^4}$ ($w = 3$) while we work in $\mathbb{Z}^*_{N^2}$ ($w = 1$), merely counting the number of group multiplications or exponentiations is not enough to give accurate performance estimates. We thus make a simplifying assumption, that group multiplication of m-bit numbers takes time $c_{\mathsf{mul}} \cdot m^{e_{\mathsf{mul}}}$ for some constants c_{mul} and e_{mul} (e.g., $e_{\mathsf{mul}} = \lg(3) \approx 1.58$ for the Karatsuba [26] algorithm), and that modular exponentiations are implemented using a square-and-multiply-like algorithm, potentially using sliding window optimizations, whose time grows linearly in the length of the exponent (whose bits are roughly a uniform distribution over $\{0, 1\}$). This gives a concrete exponentiation cost estimate of $c_{\mathsf{exp}} \cdot l \cdot m^{e_{\mathsf{mul}}}$ where l is the exponent length in bits.

We present the analytical running time for each significant MKHSS procedure in Table 3. We include only group operations and prime generation time, and ignore symmetric-key operations since their cost is negligible. Whenever there is at least one modular exponentiation, we neglect the cost of a few extra group multiplication operations. The cost of exponentiation base-$(N + 1)$ is likewise negligible because of the optimization based on binomial expansion. We use $\lg$ to denote the base-2 logarithm.

4.2.2. Safe Prime Generation. We require generating two safe primes as part of the procedure to generate a common reference string (CRS). We use an optimized implementation that trial divides both p and $(p - 1)/2$ before modular-exponentiation-based primality tests. Recall the CRS only needs to be generated once, and can be used by all future users of the protocol. For completeness, we run 10 trials for the procedure to generate an l_N-bit modulus as a product of two safe primes and report the quartiles in Table 4.

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

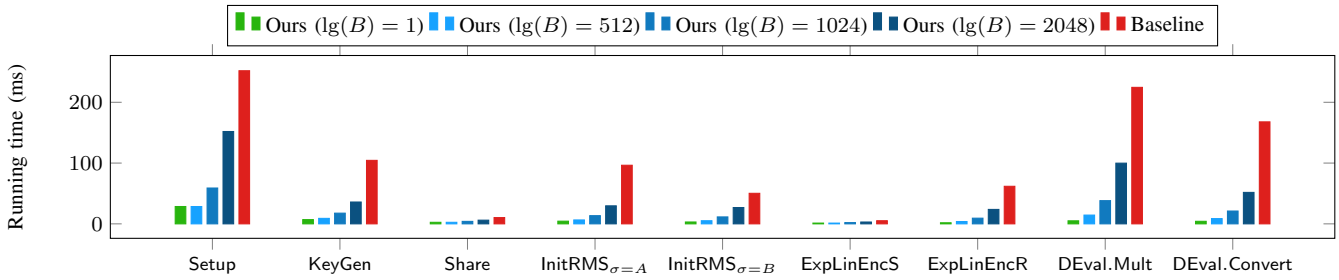


Figure 2: The runtime of our scheme versus the baseline for various $\lg(B)$. See Table 1 for the meaning of labels.

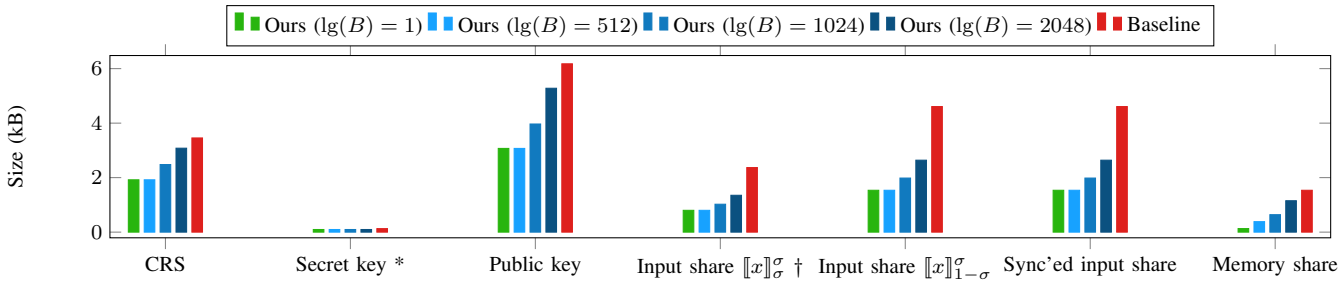


Figure 3: The size of keys and encodings for our MKHSS scheme versus the baseline.

*: For both our scheme and the baseline, since the encryption secret is of the form $s = tX + 1$, we only store t to save space.
†: For the baseline, $[\![x]\!]^{\sigma}_{\sigma}$ contains x in plaintext, so its size is dependent on $|x|$. We pick $|x| = 1$ to give the baseline an advantage.

TABLE 3: The analytical running time of significant procedures for this MKHSS scheme versus the baseline. We assume the Karatsuba multiplication exponent $e_{\mathsf{mul}} = \lg(3)$, and let $T_{\mathsf{exp}}(m) = 3 \cdot c_{\mathsf{exp}} \cdot m^{\lg(3)}$.

Procedure	Analytical runtime		Speedup for $\lg(B) = 1$		Speedup for $\lg(B) = 512$	
	Ours	Baseline	Analytical	Empirical	Analytical	Empirical
Encode an input share	$4\lambda \cdot T_{\mathsf{exp}}(l_N)$	$16\lambda \cdot T_{\mathsf{exp}}(l_N)$	4.0×	4.0×	4.0×	4.0×
Sync sender input share	$2\lambda \cdot T_{\mathsf{exp}}(l_N)$	$8\lambda \cdot T_{\mathsf{exp}}(l_N)$	4.0×	4.0×	4.0×	4.0×
Sync received input share	$(\lg(B) + \tau + 2\lambda) \cdot T_{\mathsf{exp}}(l_N)$	$4(l_N + 2\lambda) \cdot T_{\mathsf{exp}}(l_N)$	35×	35×	15×	16×
Multiply memory and input	$(4\lg(B) + 5\tau + 3\lambda) \cdot T_{\mathsf{exp}}(l_N)$	$16 l_N T_{\mathsf{exp}}(l_N)$	48×	45×	16×	16×

TABLE 4: The quartiles of prime generation for various l_N.

l_N	First quartile	Second quartile (median)	Third quartile
3072	1.3 s	1.8 s	3.3 s
3520	1.6 s	3.6 s	6.6 s
3968	3.8 s	5.0 s	8.4 s

4.3. Application: Geolocation-Based Key Exchange

We report the performance of applying our MKHSS scheme to the geolocation-based key exchange below.

Parameters. The geolocation-based key exchange models a location x_σ as a D-dimensional coordinate of L-bit integers, and key exchange is successful if the distance between x_A and x_B is less than a threshold d under the Chebyshev distance metric.

We support arbitrary distance thresholds d, whose value does not affect our running time, so we pick d at random for evaluation.

Terminology. Recall that since our scheme is an instantiation of ANIKE, and we refer the reader to Figure 1 for the communication pattern of the ANIKE protocol. To summarize, each party publishes a public encoding based on their geolocation using GeoKE.AttrKeyGen, and the other party downloads the public encoding and runs GeoKE.AttrKeyDer to derive a key, and the two keys agree only if the locations are close. Each party needs to store some amount of private state in between the two calls.

The geolocation-based key exchange is asymmetric in the sense that one party (who can specify the distance threshold d) has a larger public encoding and a longer running time for GeoKE.AttrKeyGen. We denote this party as Bob in our evaluation.

Result. We present the running time for each procedure for various choices of D and L in Table 5. $D = 2$ and $L = 32$ gives a key exchange based on real-world geographic coordinates (i.e., latitude and longitude) at the precision of at least a centimeter. $D = 3$ allows incorporating elevation,

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

TABLE 5: Comparing runtime of geolocation-based key exchange using different MKHSS schemes on our *laptop*.

	$D = 2, L = 32$			$D = 3, L = 48$			$D = 4, L = 64$		
Procedure	Ours	Baseline	Our speedup	Ours	Baseline	Our speedup	Ours	Baseline	Our speedup
GeoKE.Setup *	30.8 ms	258 ms	8.4×	29.2 ms	256 ms	8.8×	29.3 ms	256 ms	8.7×
GeoKE.AttrKeyGen($\sigma = A$)	64.0 ms	256 ms	4.0×	103 ms	391 ms	3.8×	163 ms	566 ms	3.5×
GeoKE.AttrKeyGen($\sigma = B$)	91.8 ms	355 ms	3.9×	166 ms	580 ms	3.5×	269 ms	902 ms	3.4×
GeoKE.AttrKeyDer($\sigma = A$)	1.56 s	53.7 s	34×	3.47 s	121 s	35×	6.19 s	216 s	35×
GeoKE.AttrKeyDer($\sigma = B$)	1.47 s	50.6 s	34×	3.24 s	113 s	35×	5.76 s	203 s	35×
A complete key exchange †	1.65 s	54.1 s	33×	3.63 s	122 s	33×	6.46 s	216 s	33×

*: Excludes prime generation time of MKHSS.
†: The longest time for any party to complete AttrKeyGen followed by the longest time for any party to complete AttrKeyDer. This is the total time to complete a key exchange excluding communication time. We exclude time for Setup because it is a one-time cost and the CRS can be reused.

TABLE 6: Communication (public encodings) and local storage (private state) requirement of a geolocation-based key exchange using our MKHSS scheme versus the baseline on our *laptop*.

	$D = 2, L = 32$			$D = 3, L = 48$			$D = 4, L = 64$		
Data	Ours	Baseline	Savings	Ours	Baseline	Savings	Ours	Baseline	Savings
Common reference string crs	1.9 kB	3.5 kB	1.8×	1.9 kB	3.5 kB	1.8×	1.9 kB	3.5 kB	1.8×
Alice's public encoding pe_A	101.4 kB	301.1 kB	3.0×	224.3 kB	669.8 kB	3.0×	396.3 kB	1185.9 kB	3.0×
Bob's public encoding pe_B	199.7 kB	596.0 kB	3.0×	445.5 kB	1333.3 kB	3.0×	789.5 kB	2365.5 kB	3.0×
Alice's private state st_A	51.3 kB	151.8 kB	3.0×	115.3 kB	341.3 kB	3.0×	204.9 kB	606.6 kB	3.0×
Bob's private state st_B	102.5 kB	303.4 kB	3.0×	230.5 kB	682.4 kB	3.0×	409.7 kB	1213.1 kB	3.0×
Total communication *	301.1 kB	897.2 kB	3.0×	669.8 kB	2003.1 kB	3.0×	1185.9 kB	3551.4 kB	3.0×

*: The sum of the sizes of pe_A and pe_B, which need to be communicated.

and it can also be used to account for the curvature of the Earth over large distances. Lastly, $D = 4$ accounts for the possibility of additional numerical requirements for a policy, such as an acceptable price per night using the homestay example introduced in Section 3. Among all parameter regimes, we show a consistent speedup of at least $40\times$ over the baseline when used to instantiate this application.

Finally, the sizes of public encodings and private states, which determine communication and local storage requirements, are in Table 6. We show a consistent $3\times$ savings over the baseline.

4.4. Application: Fuzzy PAKE

We report the performance of applying our MKHSS scheme to fuzzy PAKE below.

Parameters. Recall that the parameter set for our fuzzy PAKE scheme is a 5-tuple (L, W, b, Q, T):

1) L: The length of the passphrase.
2) W: The maximum length of a word in characters.
3) b: The number of bits per character.
4) Q: The number of character substitutions permitted in a word.
5) T: The number of incorrect words allowed.

Result. We provide the running time of fPAKE.AttrKeyGen and fPAKE.AttrKeyDer in Table 7 for a variety of parameters, and show a consistent speedup over the baseline:

1) $(L, W, b, Q, T) = (8, 9, 5, 2, 2)$, which is sufficient for a secure passphrase with words from Electronic Frontier Foundation's large word list with appropriate filtering, see Section 4.4.1.
2) $(L, W, b, Q, T) = (10, 8, 16, 1, 1)$, which works with 10-word UTF-16 passphrases with up to 8-characters per word at a typo tolerance of 1 character per word and 1 incorrect word.
3) $(L, W, b, Q, T) = (12, 10, 8, 3, 3)$, which works with 12-word (extended) ASCII passphrases with up to 10-characters per word at a typo tolerance of 3 characters per word and 3 words.

Lastly, we give the sizes of public encodings and private states in Table 8.

4.4.1. Demonstrating practicality with real passphrases. For the parameter set $(L, W, b, Q, T) = (8, 9, 5, 2, 2)$, An adversary has at most $2^{-51.1}$ probability of guessing a particular passphrase, which is comparable to that of a four-word passphrase under exact matching ($2^{-51.7}$). In the interest of space, we defer the security analysis we used to compute these probabilities to the full version [25].

4.5. Further Acceleration Using Vectorization

Using efficient vectorized modular multiplication techniques [34], our scheme can be further accelerated on

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

TABLE 7: The running time of fuzzy PAKE using our MKHSS scheme versus the baseline on our *laptop*.

	$(L, W, b, Q, T) = (8, 9, 5, 2, 2)$			$(L, W, b, Q, T) = (10, 8, 16, 1, 1)$			$(L, W, b, Q, T) = (12, 10, 8, 3, 3)$		
Procedure	Ours	Baseline	Our speedup	Ours	Baseline	Our speedup	Ours	Baseline	Our speedup
fPAKE.Setup	30.4 ms	270 ms	8.9×	31.1 ms	257 ms	8.3×	30.9 ms	260 ms	8.4×
fPAKE.AttrKeyGen($\sigma = A$)	214 ms	700 ms	3.3×	574 ms	1.70 s	3.0×	455 ms	1.37 s	3.0×
fPAKE.AttrKeyGen($\sigma = B$)	212 ms	671 ms	3.2×	557 ms	1.69 s	3.0×	450 ms	1.35 s	3.0×
fPAKE.AttrKeyDer($\sigma = A$)	7.35 s	252 s	34×	19.1 s	675 s	35×	27.0 s	919 s	34×
fPAKE.AttrKeyDer($\sigma = B$)	7.27 s	251 s	35×	19.2 s	676 s	35×	27.0 s	917 s	34×
A complete key exchange	7.56 s	252 s	33×	19.7 s	678 s	34×	27.5 s	920 s	33×

TABLE 8: Communication and local storage requirement of a fuzzy PAKE using our MKHSS scheme versus the baseline on our *laptop*. Since PAKE is symmetric, the results are the same for both Alice and Bob.

	$(L, W, b, Q, T) = (8, 9, 5, 2, 2)$			$(L, W, b, Q, T) = (10, 8, 16, 1, 1)$			$(L, W, b, Q, T) = (12, 10, 8, 3, 3)$		
Data	Ours	Baseline	Savings	Ours	Baseline	Savings	Ours	Baseline	Savings
Common reference string crs	1.9 kB	3.5 kB	1.8×	1.9 kB	3.5 kB	1.8×	1.9 kB	3.5 kB	1.8×
Public encoding pe	556.1 kB	1665.1 kB	3.0×	1969.2 kB	5904.4 kB	3.0×	1477.7 kB	4429.9 kB	3.0×
Private state st	556.1 kB	1665.1 kB	3.0×	1969.2 kB	5904.4 kB	3.0×	1477.7 kB	4429.9 kB	3.0×
Total communication	1.1 MB	3.3 MB	3.0×	3.9 MB	11.8 MB	3.0×	3.0 MB	8.9 MB	3.0×

TABLE 9: The running time of geolocation-based key exchange using AVX512-accelerated modular multiplication by Langowski and Devadas [34] versus just using FLINT [35] on our *AWS* machine, using our optimized MKHSS scheme.

	$D = 2, L = 32$			$D = 3, L = 48$			$D = 4, L = 64$		
Procedure	AVX512	FLINT	Speedup	AVX512	FLINT	Speedup	AVX512	FLINT	Speedup
GeoKE.Setup	16.6 ms	55.9 ms	3.4×	16.8 ms	55.8 ms	3.3×	17.2 ms	55.8 ms	3.2×
GeoKE.AttrKeyGen($\sigma = A$)	24.5 ms	78.1 ms	3.2×	42.7 ms	130 ms	3.0×	66.6 ms	204 ms	3.1×
GeoKE.AttrKeyGen($\sigma = B$)	37.8 ms	119 ms	3.2×	68.7 ms	210 ms	3.1×	110 ms	336 ms	3.1×
GeoKE.AttrKeyDer($\sigma = A$)	660 ms	2.27 s	3.4×	1.50 s	5.08 s	3.4×	2.66 s	9.02 s	3.4×
GeoKE.AttrKeyDer($\sigma = B$)	609 ms	2.08 s	3.4×	1.38 s	4.64 s	3.4×	2.46 s	8.24 s	3.3×
A complete key exchange	698 ms	2.39 s	3.4×	1.57 s	5.29 s	3.4×	2.77 s	9.35 s	3.4×

TABLE 10: Similar AVX512 versus FLINT comparison (in the same manner as Table 9) for fuzzy PAKE.

	$(L, W, b, Q, T) = (8, 9, 5, 2, 2)$			$(L, W, b, Q, T) = (10, 8, 16, 1, 1)$			$(L, W, b, Q, T) = (12, 10, 8, 3, 3)$		
Procedure	AVX512	FLINT	Speedup	AVX512	FLINT	Speedup	AVX512	FLINT	Speedup
fPAKE.Setup	16.6 ms	55.6 ms	3.4×	16.9 ms	55.6 ms	3.3×	16.6 ms	55.7 ms	3.4×
fPAKE.AttrKeyGen($\sigma = A$)	82.8 ms	255 ms	3.1×	227 ms	669 ms	2.9×	182 ms	537 ms	2.9×
fPAKE.AttrKeyGen($\sigma = B$)	81.1 ms	251 ms	3.1×	222 ms	668 ms	3.0×	179 ms	537 ms	3.0×
fPAKE.AttrKeyDer($\sigma = A$)	3.09 s	10.0 s	3.2×	8.24 s	26.7 s	3.2×	11.5 s	36.6 s	3.2×
fPAKE.AttrKeyDer($\sigma = B$)	3.09 s	9.99 s	3.2×	8.21 s	26.7 s	3.2×	11.5 s	36.5 s	3.2×
A complete key exchange	3.17 s	10.3 s	3.2×	8.47 s	27.4 s	3.2×	11.7 s	37.2 s	3.2×

machines that support the AVX512-IFMA instruction set. We compare the running times of our two key exchange applications on the *AWS* machine with and *without* the use of Langowski and Devadas [34] in Table 9 and Table 10.

5. Related Works

Both multi-key HSS and attribute-based non-interactive key exchange are new notions that distinguish themselves from prior work in many aspects. We compare with prior work to further highlight the differences.

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

Drawbacks of weaker notions of HSS. We stress that this is the first practical implementation of *multi-key* HSS. All prior constructions achieve at best the weaker notion of public-key HSS [3], which cannot be used to cleanly instantiate our applications to key exchange without resorting to extra trust assumptions, the need for a correlated setup, and extra rounds of communication.

Additional differences from Boyle et al. Boyle et al. [4] provided a concrete implementation of public-key HSS with worse security guarantees. Unlike our work, they have a high and non-negligible failure probability (on the order of 2^{-5} to 2^{-15} for practical parameters), which leaks inputs and keys via a selective-failure attack. In addition, they evaluated on an implementation that only achieves 80 bits of security (while we evaluate on 128 bits, the standard level to defend against all realistic adversaries). Despite their efforts to reduce the leakage via leakage-absorbing pads, they (and all known constructions based on DDH) cannot circumvent the barrier [36] that a failure probability under ε *requires* blowing up computation to $\Omega(1/\sqrt{\varepsilon})$.

Alternative Approaches to Key Exchange. To the best of our knowledge, all prior approaches to attribute-based key exchange require interaction [10], [11], [12], i.e., at least one party needs to receive a message from the other party *before* it can encode its own inputs. In addition to performance overheads of communication, interactivity complicates the security proof, and many schemes (e.g., [10]) are broken due to subtle flaws [37].

One further difference with [11] is that they require a trusted authority to vouch for each client's input attributes. In contrast, ANIKE only requires a single reusable CRS generated in any trusted manner, including via a secure multi-party computation. In addition, our inputs can be self-reported, which is especially relevant for sensitive information like passphrases and travel destinations.

6. Conclusion

We presented a concretely-efficient construction of multi-key homomorphic secret sharing, a recently introduced primitive that can be applied to a rich class of attribute-based non-interactive key exchange protocols. Unlike prior approaches to HSS and key exchange, MKHSS and ANIKE allows both parties to encode their inputs immediately without prior messages from another party. We believe our algorithmic insights that made our scheme practical may be of independent interest in single-key HSS as well. We identified several practically useful scenarios for key exchange, including the use of geolocation and passphrases, and believe that our work serves to bring HSS to practical realms.

Acknowledgments

We would like to thank Simon Langowski for helpful discussions around hardware acceleration using his algorithm [34] and implementation, and anonymous reviewers as well as the shepherd for their valuable comments. Kaiwen He and Srinivas Devadas acknowledge the support of the National Science Foundation (NSF). This material is based upon work supported by the NSF under Award No. 2330065. Geoffroy Couteau acknowledges the support of the French Agence Nationale de la Recherche (ANR), under the France 2030 ANR Project ANR-22-PECY003 SecureCompute. This work is supported by ERC grant OBELiSC (101115790).

References

[1] E. Boyle, N. Gilboa, and Y. Ishai, "Breaking the circuit size barrier for secure computation under DDH," in *CRYPTO 2016, Part I*, ser. LNCS, M. Robshaw and J. Katz, Eds., vol. 9814. Springer, Berlin, Heidelberg, Aug. 2016, pp. 509–539.

[2] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *41st ACM STOC*, M. Mitzenmacher, Ed. ACM Press, May / Jun. 2009, pp. 169–178.

[3] E. Boyle, N. Gilboa, and Y. Ishai, "Group-based secure computation: Optimizing rounds, communication, and computation," in *EUROCRYPT 2017, Part II*, ser. LNCS, J.-S. Coron and J. B. Nielsen, Eds., vol. 10211. Springer, Cham, Apr. / May 2017, pp. 163–193.

[4] E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, and M. Orrù, "Homomorphic secret sharing: Optimizations and applications," in *ACM CCS 2017*, B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, Eds. ACM Press, Oct. / Nov. 2017, pp. 2105–2122.

[5] D. Abram, I. Damgård, C. Orlandi, and P. Scholl, "An algebraic framework for silent preprocessing with trustless setup and active security," in *CRYPTO 2022, Part IV*, ser. LNCS, Y. Dodis and T. Shrimpton, Eds., vol. 13510. Springer, Cham, Aug. 2022, pp. 421–452.

[6] G. Couteau, L. Devadas, A. Hegde, A. Jain, and S. Servan-Schreiber, "Multi-key homomorphic secret sharing," in *EUROCRYPT 2025, Part V*, ser. LNCS, S. Fehr and P.-A. Fouque, Eds., vol. 15605. Springer, Cham, May 2025, pp. 3–33.

[7] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *EUROCRYPT'99*, ser. LNCS, J. Stern, Ed., vol. 1592. Springer, Berlin, Heidelberg, May 1999, pp. 223–238.

[8] E. S. V. Freire, D. Hofheinz, E. Kiltz, and K. G. Paterson, "Non-interactive key exchange," in *PKC 2013*, ser. LNCS, K. Kurosawa and G. Hanaoka, Eds., vol. 7778. Springer, Berlin, Heidelberg, Feb. / Mar. 2013, pp. 254–271.

[9] W. Diffie and M. E. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976.

[10] P.-A. Dupont, J. Hesse, D. Pointcheval, L. Reyzin, and S. Yakoubov, "Fuzzy password-authenticated key exchange," in *EUROCRYPT 2018, Part III*, ser. LNCS, J. B. Nielsen and V. Rijmen, Eds., vol. 10822. Springer, Cham, Apr. / May 2018, pp. 393–424.

[11] V. Kolesnikov, H. Krawczyk, Y. Lindell, A. J. Malozemoff, and T. Rabin, "Attribute-based key exchange with general policies," in *ACM CCS 2016*, E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, Eds. ACM Press, Oct. 2016, pp. 1451–1463.

[12] J. Bootle, S. H. Faller, J. Hesse, K. Hostáková, and J. Ottenhues, "Generalized fuzzy password-authenticated key exchange from error correcting codes," in *ASIACRYPT 2023, Part VIII*, ser. LNCS, J. Guo and R. Steinfeld, Eds., vol. 14445. Springer, Singapore, Dec. 2023, pp. 110–142.

[13] S. M. Bellovin and M. Merritt, "Encrypted key exchange: Password-based protocols secure against dictionary attacks," in *1992 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 1992, pp. 72–84.

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

[14] S. Halevi, Y. Lindell, and B. Pinkas, "Secure computation on the web: Computing without simultaneous interaction," in *CRYPTO 2011*, ser. LNCS, P. Rogaway, Ed., vol. 6841. Springer, Berlin, Heidelberg, Aug. 2011, pp. 132–150.

[15] T. Koshiba and K. Kurosawa, "Short exponent Diffie-Hellman problems," in *PKC 2004*, ser. LNCS, F. Bao, R. Deng, and J. Zhou, Eds., vol. 2947. Springer, Berlin, Heidelberg, Mar. 2004, pp. 173–186.

[16] M. Adjedj, C. Blokh, G. Couteau, A. Galansky, A. Joux, and N. Makriyannis, "Two-round 2PC ECDSA at the cost of 1 OLE," Cryptology ePrint Archive, Report 2024/1950, 2024. [Online]. Available: https://eprint.iacr.org/2024/1950

[17] J. M. Pollard, "Monte carlo methods for index computation ($\mathrm{mod}\ p$)," *Mathematics of computation*, vol. 32, no. 143, pp. 918–924, 1978.

[18] G. Couteau, M. Klooß, H. Lin, and M. Reichle, "Efficient range proofs with transparent setup from bounded integer commitments," in *EUROCRYPT 2021, Part III*, ser. LNCS, A. Canteaut and F.-X. Standaert, Eds., vol. 12698. Springer, Cham, Oct. 2021, pp. 247–277.

[19] G. Couteau, D. Goudarzi, M. Klooß, and M. Reichle, "Sharp: Short relaxed range proofs," in *ACM CCS 2022*, H. Yin, A. Stavrou, C. Cremers, and E. Shi, Eds. ACM Press, Nov. 2022, pp. 609–622.

[20] D. K. Gillmor, "Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security (TLS)," RFC 7919, Aug. 2016. [Online]. Available: https://www.rfc-editor.org/info/rfc7919

[21] M. Friedl, N. Provos, and W. A. Simpson, "Diffie-Hellman Group Exchange for the Secure Shell (SSH) Transport Layer Protocol," RFC 4419, Mar. 2006. [Online]. Available: https://www.rfc-editor.org/info/rfc4419

[22] L. Valenta, D. Adrian, A. Sanso, S. Cohney, J. Fried, M. Hastings, J. A. Halderman, and N. Heninger, "Measuring small subgroup attacks against Diffie-Hellman," in *NDSS 2017*. The Internet Society, Feb. / Mar. 2017.

[23] E. Bresson, D. Catalano, and D. Pointcheval, "A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications," in *ASIACRYPT 2003*, ser. LNCS, C.-S. Laih, Ed., vol. 2894. Springer, Berlin, Heidelberg, Nov. / Dec. 2003, pp. 37–54.

[24] C. Orlandi, P. Scholl, and S. Yakoubov, "The rise of paillier: Homomorphic secret sharing and public-key silent OT," in *EUROCRYPT 2021, Part I*, ser. LNCS, A. Canteaut and F.-X. Standaert, Eds., vol. 12696. Springer, Cham, Oct. 2021, pp. 678–708.

[25] K. He, S. Servan-Schreiber, G. Couteau, and S. Devadas, "Concretely-efficient multi-key homomorphic secret sharing and applications," Cryptology ePrint Archive, Paper 2025/1803, 2025. [Online]. Available: https://eprint.iacr.org/2025/1803

[26] A. A. Karatsuba and Y. P. Ofman, "Multiplication of many-digital numbers by automatic computers," in *Doklady Akademii Nauk*, vol. 145, no. 2. Russian Academy of Sciences, 1962, pp. 293–294.

[27] E. Boyle, L. Devadas, and S. Servan-Schreiber, "Non-interactive distributed point functions," in *PKC 2025, Part I*, ser. LNCS, T. Jager and J. Pan, Eds., vol. 15674. Springer, Cham, May 2025, pp. 3–35.

[28] W. Deng, B. Zhao, Y. Xiao, Y. Zhong, Q. Pei, and X. Liu, "MORSE: An efficient homomorphic secret sharing scheme enabling non-linear operation," *arXiv preprint arXiv:2410.06514*, 2024.

[29] M. Bellare, D. Pointcheval, and P. Rogaway, "Authenticated key exchange secure against dictionary attacks," in *EUROCRYPT 2000*, ser. LNCS, B. Preneel, Ed., vol. 1807. Springer, Berlin, Heidelberg, May 2000, pp. 139–155.

[30] E. Protalinski, "Facebook passwords are not case sensitive," http://www.zdnet.com/article/facebook-passwords-are-not-case-sensitive-update/, 2015, accessed: 2025-09-28.

[31] R. Chatterjee, A. Athayle, D. Akhawe, A. Juels, and T. Ristenpart, "pASSWORD tYPOS and how to correct them securely," in *2016 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2016, pp. 799–818.

[32] I. Damgård, M. Jurik, and J. B. Nielsen, "A generalization of Paillier's public-key system with applications to electronic voting," *International Journal of Information Security*, vol. 9, pp. 371–385, 2010.

[33] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*, ser. The CRC Press series on discrete mathematics and its applications. 2000 N.W. Corporate Blvd., Boca Raton, FL 33431-9868, USA: CRC Press, 1997.

[34] S. Langowski and S. Devadas, "Efficient modular multiplication using vector instructions on commodity hardware," Cryptology ePrint Archive, Report 2025/1068, 2025. [Online]. Available: https://eprint.iacr.org/2025/1068

[35] T. F. team, *FLINT: Fast Library for Number Theory*, 2025, version 3.2.2, https://flintlib.org.

[36] I. Dinur, N. Keller, and O. Klein, "An optimal distributed discrete log protocol with applications to homomorphic secret sharing," in *CRYPTO 2018, Part III*, ser. LNCS, H. Shacham and A. Boldyreva, Eds., vol. 10993. Springer, Cham, Aug. 2018, pp. 213–242.

[37] J. Januzelli, L. Roy, and J. Xu, "Under what conditions is encrypted key exchange actually secure?" in *EUROCRYPT 2025, Part II*, ser. LNCS, S. Fehr and P.-A. Fouque, Eds., vol. 15602. Springer, Cham, May 2025, pp. 451–481.

[38] I. Damgård and M. Jurik, "A length-flexible threshold cryptosystem with applications," in *ACISP 03*, ser. LNCS, R. Safavi-Naini and J. Seberry, Eds., vol. 2727. Springer, Berlin, Heidelberg, Jul. 2003, pp. 350–364.

[39] I. Damgård and M. Jurik, "A generalisation, a simplification and some applications of Paillier's probabilistic public-key system," in *PKC 2001*, ser. LNCS, K. Kim, Ed., vol. 1992. Springer, Berlin, Heidelberg, Feb. 2001, pp. 119–136.

[40] L. Roy and J. Singh, "Large message homomorphic secret sharing from DCR and applications," in *CRYPTO 2021, Part III*, ser. LNCS, T. Malkin and C. Peikert, Eds., vol. 12827. Virtual Event: Springer, Cham, Aug. 2021, pp. 687–717.

[41] E. Boyle, N. Gilboa, Y. Ishai, H. Lin, and S. Tessaro, "Foundations of homomorphic secret sharing," in *ITCS 2018*, A. R. Karlin, Ed., vol. 94. LIPIcs, Jan. 2018, pp. 21:1–21:21.

Appendix A. Preliminaries

We denote $\mathsf{negl}(\cdot)$ a negligible function: a function that is asymptotically smaller than any inverse polynomial.

A.1. Dåmgard-Jurik-ElGamal Encryption

Dåmgard-Jurik-ElGamal Encryption (DJEG) [6], [38] is a natural generalization of Paillier-ElGamal [23] encryption. We sketch a variant of the general DJEG algorithm secure assuming the short-exponent indistinguishability (Section 2) assumption: The common reference string consists of N, a random product of two safe primes, and g, a random generator of the group of $2N^w$-th residues in $\mathbb{Z}^*_{N^{w+1}}$. Unlike RSA, DJEG remains secure if the same N is used by all parties. To generate a key, each party samples its secret key as $s \leftarrow\$ [2^{2\lambda}]$, which defines the public key $f = g^{-s} \in \mathbb{Z}^*_{N^{w+1}}$.

To encrypt a plaintext $x \in \mathbb{Z}^*_{N^w}$, a party samples $r \leftarrow\$ [2^{2\lambda}]$ and then outputs $\mathsf{ct} := (g^r, f^r(1+N)^x)$. To decrypt

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

$\mathsf{ct} = (\mathsf{ct}_0, \mathsf{ct}_1)$, the decryptor computes $z := \mathsf{ct}_0^s \mathsf{ct}_1 = (1+N)^x$ and computes the discrete logarithm of $z \in \mathbb{Z}^*_{N^{w+1}}$ with respect to $1+N$ to recover the message x. This can be done via $x = \frac{z-1}{N}$ [7] when $w = 1$, and the generalized algorithm for $w > 1$, which we denote as $L_w(\cdot)$, is in [39].

A.2. Distributed Discrete Log

The distributed discrete log procedure [24], [40] $\mathsf{DDLog}_{N,w}(\cdot)$ converts a *divisive* share of $(1+N)^x \in \mathbb{Z}^*_{N^{w+1}}$ to a subtractive share of x *over the group* $\mathbb{Z}_{N^w}$. More precisely, we have that for any $g' \in \mathbb{Z}^*_{N^{w+1}}$,

$$\mathsf{DDLog}_{N,w}(g'(1+N)^x) - \mathsf{DDLog}_{N,w}(g') \equiv x \bmod N^w.$$

Since our MKHSS construction requires subtractive shares over the *integers*, two parties often apply a common random shift $r \in [N^w]$ to the output of DDLog to obtain a subtractive share over $\mathbb{Z}$ with high probability over r: $\langle z \rangle_\sigma \leftarrow \mathsf{DDLog}_{N,w}(g^{\langle 0 \rangle_\sigma}(1+N)^{\langle z \rangle_\sigma}) + r \bmod N^w$.

We sketch an implementation using $L_w(\cdot)$, which computes the discrete log with base $1+N$, as a subroutine:

$$\begin{aligned}&\mathsf{DDLog}_{N,w}(h \in \mathbb{Z}^*_{N^{w+1}})\\ &:= L_w(h(h \bmod N)^{-1} \bmod N^{w+1}).\end{aligned}$$

A proof of correctness for this can be found in [5].

A.3. Non-Interactive Multiplication

Non-interactive multiplication (NIM) [27] allows Alice and Bob, holding t_A and t_B, to exchange a simultaneous message and each obtain a subtractive share of the product $\langle t_A t_B \rangle_\sigma$. We give a simple construction below, assuming that both parties have a shared random value $r_{\mathsf{nim}} \in [N^w]$:

$\mathsf{NIM.Setup}(1^\lambda) \to \mathsf{crs}^{\mathsf{nim}}$ samples two random $2N^w$-th residues $g, h \in \mathbb{Z}^*_{N^{w+1}}$ and outputs $\mathsf{crs}^{\mathsf{nim}} := (g, h)$

$\mathsf{NIM.Encode}(\mathsf{crs}^{\mathsf{nim}}, \sigma, t_\sigma) \to (\mathsf{pe}^{\mathsf{nim}}_\sigma, \mathsf{st}^{\mathsf{nim}}_\sigma)$ is defined differently based on $\sigma \in \{A, B\}$:

1) For $\sigma = A$, sample $r_A \leftarrow\$ [2^{2\lambda}]$, and set $\mathsf{pe}^{\mathsf{nim}}_A := g^{r_A} h^{t_A}$ and $\mathsf{st}^{\mathsf{nim}}_\sigma := (t_A, r_A)$.
2) For $\sigma = B$, sample $r_B \leftarrow\$ [2^{2\lambda}]$, and set $\mathsf{pe}^{\mathsf{nim}}_B := (g^{r_B}, (N+1)^{t_B} h^{r_B})$ and $\mathsf{st}^{\mathsf{nim}}_\sigma := r_B$.

$\mathsf{NIM.Decode}(\mathsf{crs}, \sigma, \mathsf{st}^{\mathsf{nim}}, \mathsf{pe}^{\mathsf{nim}}_{1-\sigma}; r_{\mathsf{nim}}) \to \langle t_A t_B \rangle_\sigma$

1 : **parse** $\mathsf{st}^{\mathsf{nim}} = (t_\sigma, r_\sigma)$
2 : **if** $\sigma = A$, **parse** $\mathsf{pe}^{\mathsf{nim}}_B = (c_0, c_1)$
3 : $Z_\sigma := c_0^{r_A} c_1^{t_A}$ **if** $\sigma = A$ **else** $(\mathsf{pe}^{\mathsf{nim}}_A)^{r_B}$
4 : **return** $\mathsf{DDLog}_{N,w}(Z_\sigma) + r_{\mathsf{nim}} \bmod N^w$

Since $Z_B = c_0^{r_A} c_1^{t_A} = g^{r_A r_B}(N+1)^{t_A t_B} h^{t_A r_B} = Z_A \cdot (N+1)^{t_A t_B}$, correctness holds with probability at least $1 - 2^{-\tau}$ over r_{nim} as long as $|t_A t_B| \le N^w 2^{-\tau}$. Security requires that the public encoding for two different t_σ is indistinguishable: observe that Alice's public encoding is statistically hiding similar to a Pedersen commitment, and that Bob's public encoding is a DJEG encryption with public key h. Since g, h are generated at random from the same group, any adversary that distinguishes Bob's encoding can be immediately applied to break the semantic security of DJEG, a contradiction.

Construction 3: Attribute-based non-interactive key exchange from NIKE and our MKHSS scheme.

Parameters. Let $\mathsf{MKHSS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Share}, \mathsf{Eval}, \mathsf{Eval}^*)$ be any MKHSS scheme with a modified Eval^* algorithm that outputs $\langle P(x_A, x_B) \cdot s \rangle_\sigma$. Let $\mathsf{NIKE} = (\mathsf{Setup}, \mathsf{KeyDer}, \mathsf{KeyGen})$ an arbitrary NIKE (e.g., Diffie-Hellman). Let P be the fixed predicate for this instantiation of ANIKE.

$\mathsf{ANIKE.Setup}(1^\lambda)$:
1 : $\mathsf{crs}_{\mathsf{mkhss}} \leftarrow \mathsf{MKHSS.Setup}(\lambda)$
2 : $\mathsf{crs}_{\mathsf{nike}} \leftarrow \mathsf{NIKE.Setup}(\lambda)$
3 : **return** $(\mathsf{crs}_{\mathsf{mkhss}}, \mathsf{crs}_{\mathsf{nike}})$

$\mathsf{ANIKE.AttrKeyDer}(\sigma, \mathsf{st}_\sigma, \mathsf{pe}_{1-\sigma})$:
1 : **parse** $\mathsf{pe}_{1-\sigma} = (\mathsf{pk}, \mathsf{pk}^{\mathsf{nike}}_{1-\sigma}, [\![\mathbf{x_{1-\sigma}}]\!]^{1-\sigma}_\sigma)$
2 : **parse** $\mathsf{st}_\sigma = (\mathsf{sk}, \mathsf{sk}^{\mathsf{nike}}_\sigma, [\![\mathbf{x_\sigma}]\!]^\sigma_\sigma)$
3 : $\langle P(\mathbf{x_A}, \mathbf{x_B}) \cdot s \rangle_\sigma := \mathsf{MKHSS.Eval}^*(\mathsf{crs}, \sigma, \mathsf{sk}_\sigma, \mathsf{pk}_{1-\sigma},$
4 : $\qquad [\![\mathbf{x_A}]\!]^A_\sigma, [\![\mathbf{x_B}]\!]^B_\sigma, P)$
5 : $k_{\mathsf{nike}} := \mathsf{NIKE.KeyDer}(\mathsf{pk}^{\mathsf{nike}}_{1-\sigma}, \mathsf{sk}^{\mathsf{nike}}_\sigma)$
6 : $k := H(\langle P(\mathbf{x_A}, \mathbf{x_B}) \cdot s \rangle_\sigma \| k_{\mathsf{nike}} \| P)$
7 : **return** k

$\mathsf{ANIKE.AttrKeyGen}(\mathsf{crs}, \sigma, \mathbf{x})$:
1 : **parse** $\mathsf{crs} = (\mathsf{crs}_{\mathsf{mkhss}}, \mathsf{crs}_{\mathsf{nike}})$
2 : $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{MKHSS.KeyGen}(\mathsf{crs})$
3 : $(\mathsf{pk}^{\mathsf{nike}}, \mathsf{sk}^{\mathsf{nike}}) \leftarrow \mathsf{NIKE.KeyGen}(\mathsf{crs}_{\mathsf{nike}})$
4 : $([\![\mathbf{x}]\!]^\sigma_A, [\![\mathbf{x}]\!]^\sigma_B) \leftarrow \mathsf{MKHSS.Share}(\mathsf{crs}, \sigma, \mathsf{pk}, \mathbf{x})$
5 : $\mathsf{pe} := (\mathsf{pk}, \mathsf{pk}^{\mathsf{nike}}, [\![\mathbf{x}]\!]^\sigma_{1-\sigma})$
6 : $\mathsf{st} := (\mathsf{sk}, \mathsf{sk}^{\mathsf{nike}}, [\![\mathbf{x}]\!]^\sigma_\sigma)$
7 : **return** $(\mathsf{pe}, \mathsf{st})$

Appendix B. Definitions

B.1. MKHSS

We define multi-key HSS in Definition 1. $[\![x]\!]^\sigma = ([\![x]\!]^\sigma_A, [\![x]\!]^\sigma_B)$ denotes an input share of a message x generated under the public key of party σ, and denote by $\{\!\{x\}\!\}$ a *synchronized* share of x which will be the same between two parties. We use $1-\sigma$ to denote the "other party".

Definition 1 (Multi-Key Homomorphic Secret Sharing)**.** *For a positive integer B, a B-bounded multi-key homomorphic secret sharing (MKHSS) scheme is a tuple of four polynomial time algorithms* $\mathsf{MKHSS} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Share}, \mathsf{Eval})$ *with the following syntax:*

- $\mathsf{Setup}(1^\lambda) \to \mathsf{crs}$. *On input a security parameter λ, generates a reusable common reference string* crs.
- $\mathsf{KeyGen}(\mathsf{crs}) \to (\mathsf{pk}, \mathsf{sk})$. *On input the CRS* crs*, each party independently runs this to generate its own pair of public and secret keys.*

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

$\mathsf{E}^{\mathsf{correct}}_{\mathbf{x}_A,\mathbf{x}_B}(\lambda)$

$\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda)$
for $\sigma \in \{A, B\}$,
  $(\mathsf{pk}_\sigma, \mathsf{sk}_\sigma) \leftarrow \mathsf{KeyGen}(\mathsf{crs})$
  $\left([\![\mathbf{x}_\sigma]\!]^\sigma_A, [\![\mathbf{x}_\sigma]\!]^\sigma_B\right) \leftarrow \mathsf{Share}(\mathsf{crs}, \sigma, \mathsf{pk}_\sigma, \mathbf{x}_\sigma)$
for $\sigma \in \{A, B\}$,
  $\langle z \rangle_\sigma \leftarrow \mathsf{Eval}(\mathsf{crs}, \sigma, \mathsf{sk}_\sigma, \mathsf{pk}_{1-\sigma}, [\![\mathbf{x}_A]\!]^A_\sigma, [\![\mathbf{x}_B]\!]^B_\sigma, P)$
return $\mathbb{1}(\langle z \rangle_A - \langle z \rangle_B = P(\mathbf{x}_A, \mathbf{x}_B))$

Figure 4: Correctness experiment for MKHSS.

- $\mathsf{Share}(\mathsf{crs}, \sigma, \mathsf{pk}_\sigma, x) \rightarrow ([\![x]\!]^\sigma_A, [\![x]\!]^\sigma_B)$. *On input the CRS* crs*, party identifier* $\sigma \in \{A, B\}$*, the public key* pk_σ *of the party, and a message* $x \in [-B+1, B-1]$*, output input shares* $[\![x]\!]^\sigma_A$ *and* $[\![x]\!]^\sigma_B$*. Abusing notation, we permit* Share *to be called on vectors* $\mathbf{x}$*.*
- $\mathsf{Eval}(\mathsf{crs}, \sigma, \mathsf{sk}_\sigma, \mathsf{pk}_{1-\sigma}, [\![\mathbf{x}_A]\!]^A_\sigma, [\![\mathbf{x}_B]\!]^B_\sigma, P) \rightarrow \langle z \rangle_\sigma$*. On input the CRS* crs*, a party identifier* $\sigma \in \{A, B\}$*, the party's secret key* sk_σ*, and the other party's public key* $\mathsf{pk}_{1-\sigma}$*, a list of input shares from Alice* $[\![\mathbf{x}_A]\!]^A_\sigma$*, and respectively from Bob* $[\![\mathbf{x}_B]\!]^B_\sigma$*, and an RMS program description* P*, output a subtractive share of the output* $z = P(\mathbf{x}, \mathbf{y})$*.*

In addition, MKHSS satisfies the following correctness and security properties:

Correctness. *A* B*-bounded MKHSS scheme is correct if there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for all* $\lambda \in \mathbb{N}$*, all* $(m_A + m_B)$*-input* B*-bounded RMS programs* P *whose memory values are in the range* $[-B+1, B-1]$*, and all* $\mathbf{x}_A \in [0, B-1]^{m_A}$ *and* $\mathbf{x}_B \in [0, B-1]^{m_B}$*:*

$$\Pr[\mathsf{E}^{\mathsf{correct}}_{\mathbf{x}_A,\mathbf{x}_B}(\lambda) = 0] \leq \mathsf{negl}(\lambda),$$

where $\mathbf{x}_\sigma = (x^{(1)}_\sigma, \ldots, x^{(m)}_\sigma)$*, and* $\mathsf{E}^{\mathsf{correct}}_{\mathbf{x}_A,\mathbf{x}_B}$ *is defined in Figure 4.*

Security. *An MKHSS scheme is said to be* secure *if for all efficient adversaries* $\mathcal{A}$*, there exists a negligible function* $\mathsf{negl}(\cdot)$ *such that for all* $\lambda \in \mathbb{N}$*, and all* $\sigma \in \{A, B\}$*, we have that*

$$\Pr\left[\begin{array}{rl} & \mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda) \\ & (\mathsf{pk}_\sigma, \mathsf{sk}_\sigma) \leftarrow \mathsf{KeyGen}(\mathsf{crs}) \\ b' = b \quad : & (x_0, x_1, \mathsf{state}) \leftarrow \mathcal{A}(\mathsf{crs}, \mathsf{pk}_\sigma) \\ & b \leftarrow\$ \{0,1\} \\ & ([\![x_b]\!]^\sigma_A, [\![x_b]\!]^\sigma_B) \leftarrow \mathsf{Share}(\mathsf{crs}, \sigma, \mathsf{pk}_\sigma, x_b) \\ & b' \leftarrow \mathcal{A}\left([\![x_b]\!]^\sigma_{1-\sigma}, \mathsf{state}\right) \end{array}\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda).$$

Remark 1 (Extension of security to multiple shares)**.** *It is straightforward to show that the security of MKHSS implies security under a modified game where* x_0 *and* x_1 *are replaced with two* vectors $\mathbf{x_0}$ *and* $\mathbf{x_1}$ *via a hybrid argument.*

B.2. ANIKE

We define ANIKE in Definition 2. The definition is almost the same as in Couteau et al. [6], except we allow asymmetric policies and fix the policy P as a parameter.

Definition 2 (Attribute-Based Non-Interactive Key Exchange)**.** *An attribute-based non-interactive key exchange (ANIKE) protocol with attribute spaces* $\mathcal{X}_A, \mathcal{X}_B$ *and a policy* $P : \mathcal{X}_A \times \mathcal{X}_B \rightarrow \{0, 1\}$ *consists of three efficient algorithms* $(\mathsf{Setup}, \mathsf{AttrKeyGen}, \mathsf{AttrKeyDer})$ *with the following syntax:*

- $\mathsf{Setup}(1^\lambda) \rightarrow \mathsf{crs}$*. On input a security parameter, generates a common reference string (CRS)* crs*.*
- $\mathsf{AttrKeyGen}(\mathsf{crs}, \sigma, x) \rightarrow (\mathsf{pe}_\sigma, \mathsf{st}_\sigma)$*. On input the CRS* crs*, party identifier* $\sigma \in \{A, B\}$*, and attribute* $x \in \mathcal{X}_\sigma$*, outputs the public encoding* pe_σ *and private state* st_σ*.*
- $\mathsf{AttrKeyDer}(\mathsf{crs}, \sigma, \mathsf{st}_\sigma, \mathsf{pe}_{1-\sigma}) \rightarrow k_\sigma$*. On input the CRS* crs*, the party identifier* $\sigma \in \{A, B\}$*, the party's own private state* st_σ*, the other party's public encoding* $\mathsf{pe}_{1-\sigma}$*, outputs a key* $k_\sigma \in \{0, 1\}^\lambda$*.*

In addition, ANIKE satisfies the following correctness and semi-honest security properties.

Correctness. *An ANIKE scheme is correct if there is a negligible function* $\mathsf{negl}(\cdot)$ *such that for all* $\lambda \in \mathbb{N}$*,* $x_A \in \mathcal{X}_A$*, and* $x_B \in \mathcal{X}_B$ *such that* $P(x_A, x_B) = 0$*, i.e., the policy* is *satisfied, we have*

$$\Pr\left[\begin{array}{rl} & \mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda) \\ & (\mathsf{pe}_A, \mathsf{st}_A) \leftarrow \mathsf{AttrKeyGen}(\mathsf{crs}, A, x_A) \\ k_A = k_B \quad : & (\mathsf{pe}_B, \mathsf{st}_B) \leftarrow \mathsf{AttrKeyGen}(\mathsf{crs}, B, x_B) \\ & k_A \leftarrow \mathsf{AttrKeyDer}(\mathsf{crs}, A, \mathsf{st}_A, \mathsf{pe}_B) \\ & k_B \leftarrow \mathsf{AttrKeyDer}(\mathsf{crs}, B, \mathsf{st}_B, \mathsf{pe}_A) \end{array}\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda).$$

Semi-honest security. *An ANIKE scheme is secure against semi-honest adversaries if it satisfies the three properties—key pseudorandomness, key independence, and attribute hiding—defined through security experiments* E^{KP}*,* E^{KI}*, and* E^{AH}*, respectively—in Figure 5.*

Appendix C. Full MKHSS Construction

We describe our MKHSS construction in Construction 4.

Theorem 1. *Assuming DCR and the hardness of short exponent discrete log over the the group of* $2N^w$*-th residues, our the construction in Construction 4 is a correct and secure MKHSS scheme.*

Proof. In the interest of space, we give a sketch of correctness and security here, and defer the full proof to the full version of this paper [25]. For correctness, we first show that $\{\!\{x\}\!\}$ derived via ExpLinEncS and ExpLinEncR are the same for both parties, and then show that $\{\!\{x\}\!\}$ is a DJEG ciphertext that decrypts to x under the joint secret key $s = s_A s_B$. Then, we show via induction that

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

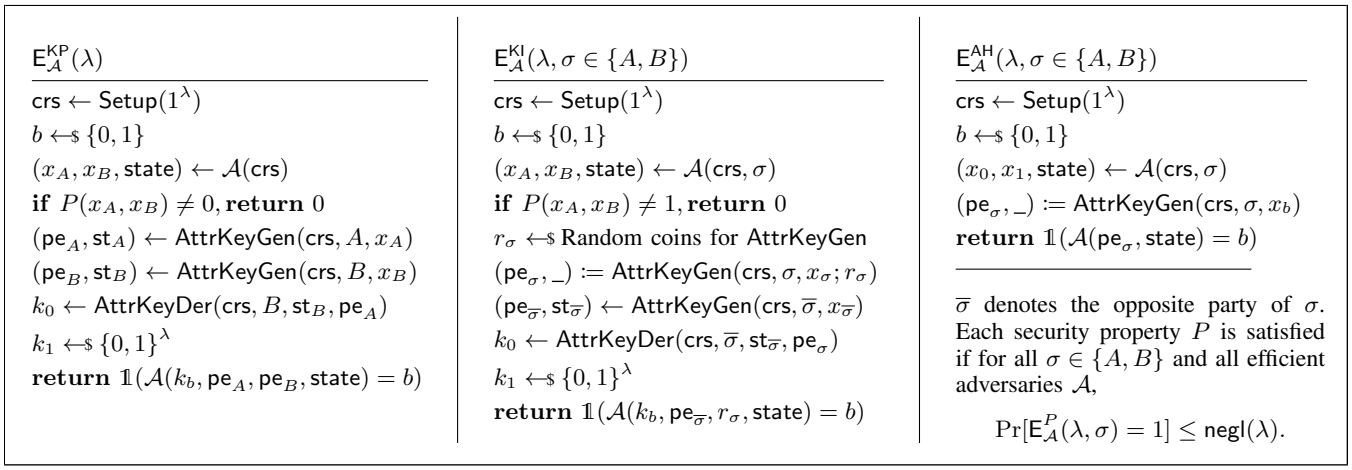


Figure 5: Security experiments for ANIKE.

all memory shares $\langle\!\langle y \rangle\!\rangle_\sigma$ consists of valid subtractive shares $\langle ys \rangle_\sigma$ and $\langle y \rangle_\sigma$. The base case follows from the correctness of NIM and our optimization to reconstruct $\langle s \rangle_\sigma$ from $\langle t_A t_B \rangle_\sigma$ and t_σ, and all later cases follows from the fact that $\{\!\{x\}\!\}$ is a DJEG ciphertext, and the fact that all memory values are B-bounded. Some care is needed to ensure that the randomization shift passed to DDLog are independent, which is achieved by using a PRF.

For security, we proceed with a sequence of hybrids from $b = 0$ to $b = 1$. In the first hybrid, we replace $\mathsf{pe}^{\mathsf{nim}}_\sigma$ with an encoding of zero, which follows from the security of NIM. Then, since t is now removed from our scheme, we change our secret key s to sample from $[N]$, via a careful use of the short-exponent indistinguishability (SEI) assumption that accounts for the structure $s = tB \cdot 2^\tau + 1$, which turns out to not affect security even in combination with SEI. Lastly, we replace the DJEG encryption of x_0 with x_1, which follows from the security of DJEG. Finally, we perform the changes in reverse order (revert secret key structure, then revert NIM to encode t_σ rather than zero. ■

Appendix D. Full ANIKE Construction

We provide the full construction in Construction 3.

Theorem 2. *Under the same assumptions that our MKHSS scheme is based on,*[5] *Construction 3 is a correct and secure ANIKE scheme.*

Proof. Correctness follows trivially from the correctness of MKHSS. For security, observe that key-pseudorandomness follows trivially from the CKS-light security [8] of NIKE since k_{nike} is hashed into the final key, and attribute-hiding follows trivially from the security of MKHSS. It remains to show the nontrivial property of key independence:

Suppose without the loss of generality that the key-independence adversary $\mathcal{A}$ wins the game with $\sigma = A$, we construct another adversary $\mathcal{A}'$ that breaks the multi-share (Remark 1) security of MKHSS for $\sigma = B$ by wrapping round $\mathcal{A}$'s input, output, and random oracle calls. Observe that for $\mathcal{A}$ to have any non-negligible advantage, it *must* query the random oracle on $\langle P(\mathbf{x_A}, \mathbf{x_B}) \cdot s \rangle_B = \langle s \rangle_B$ non-negligibly often, since otherwise k_0 and k_1 are identically distributed by the randomness of the oracle H.

First, $\mathcal{A}'$ relays crs and pass $\sigma = A$ to $\mathcal{A}$, who then responds with $\mathbf{x_A}, \mathbf{x_B}$. For $b \in \{0,1\}$, $\mathcal{A}'$ sets $\mathbf{x_b} = (\mathbf{x_B}, b)$, and sends $\mathbf{x_0}, \mathbf{x_1}$ to the MKHSS challenger and get back $\mathbf{x_b}$ parsed out as $[\![\mathbf{x_B}]\!]^B_A, [\![b]\!]^B_A$. Then, $\mathcal{A}'$ runs Alice's protocol honestly, saves the coins r_A, and generates a NIKE keypair $\mathsf{sk}_B, \mathsf{pk}_B$ for Bob. It then samples a *fake* Bob's key $k^*_b \leftarrow\$ \{0,1\}$ and invokes $\mathcal{A}(k_b, (\mathsf{pk}_B, \mathsf{pe}, [\![\mathbf{x_B}]\!]^B_A), r_A, \mathsf{state})$.

Finally, $\mathcal{A}'$ scans through the list of queries that $\mathcal{A}$ makes. For every query, it tries to parse it as $\langle s_B \rangle$, compute $s = \langle s \rangle_A - \langle s \rangle_B$, and check if $g^{-s} = f_B^{s_A}$. If so, it decrypts the DJEG ciphertext $[\![b]\!]^B_A$ and learns b. This must happen with non-negligible probability since up until the first query to the random oracle with $\langle s \rangle_B$, $\mathcal{A}$ cannot tell whether it is interacting with a real challenger or used as a subroutine of $\mathcal{A}'$ (specifically, the fake key k_b looks just like a challenger's key) due to the randomness of the oracle. ■

Appendix E. Malicious Security

Our ANIKE construction achieves security against semi-honest adversaries, as reflected in the security experiment $\mathsf{E}^{\mathsf{KI}}_{\mathcal{A}}$. In the interest of space, we show how our scheme can be upgraded to malicious security (while keeping the key exchange non-interactive) in the full version of this paper [25].

5. MKHSS implies NIKE, see [41]

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

Construction 4: Our MKHSS scheme

Parameters. Security parameter λ, statistical security parameter τ, bit length of the modulus l_N, RMS program bound B, and DJEG parameter w. $\mathsf{GenPQ}(1^\lambda)$ generates two $\lceil l_N/2 \rceil$-bit safe primes. $\mathsf{DDLog}_{N,w}$ is defined in Appendix A.2. $\mathsf{NIM} = (\mathsf{NIM.Setup}, \mathsf{NIM.Encode}, \mathsf{NIM.Decode})$ is defined in Appendix A.3. H is a random oracle. Let $G_m : \{0,1\}^\lambda \to \mathbb{Z}_m$ be a pseudorandom generator, and $F_m : \{0,1\}^\lambda \times \mathbb{Z} \to \mathbb{Z}_m$ be a pseudorandom function with output space $\mathbb{Z}_m$.

$\mathsf{MKHSS.Setup}(1^\lambda, 1^\tau)$:

1 : $(p, q) \leftarrow \mathsf{GenPQ}(1^\lambda)$
2 : $N \coloneqq pq$
3 : $g_0 \leftarrow\!\$\ \mathbb{Z}^*_{N^{w+1}}$
4 : $g \coloneqq (g_0)^{2N^w} \bmod N^{w+1}$
5 : $\mathsf{crs}^{\mathsf{nim}} \coloneqq \mathsf{NIM.Setup}(1^\lambda)$
6 : $\mathsf{crs} \coloneqq (N, g, \mathsf{crs}^{\mathsf{nim}})$
7 : **return** crs

$\mathsf{MKHSS.KeyGen}(\mathsf{crs})$:

1 : **parse** $\mathsf{crs} = (N, g, \mathsf{crs}^{\mathsf{nim}})$
2 : $t \leftarrow\!\$\ [2^{2\lambda}]$
3 : $s \coloneqq t \cdot (B \cdot 2^\tau) + 1$
4 : $f \coloneqq g^{-s} \bmod N^{w+1}$
5 : $(\mathsf{pe}^{\mathsf{nim}}_A, \mathsf{st}^{\mathsf{nim}}_A) \leftarrow \mathsf{NIM.Encode}(\mathsf{crs}^{\mathsf{nim}}, A, t)$
6 : $(\mathsf{pe}^{\mathsf{nim}}_B, \mathsf{st}^{\mathsf{nim}}_B) \leftarrow \mathsf{NIM.Encode}(\mathsf{crs}^{\mathsf{nim}}, B, t)$
7 : $\mathsf{pk} \coloneqq (f, \mathsf{pe}^{\mathsf{nim}}_A, \mathsf{pe}^{\mathsf{nim}}_B)$
8 : $\mathsf{sk} \coloneqq (s, \mathsf{st}^{\mathsf{nim}}_A, \mathsf{st}^{\mathsf{nim}}_B)$
9 : **return** $(\mathsf{pk}, \mathsf{sk})$

$\mathsf{MKHSS.Share}(\mathsf{crs}, \sigma, \mathsf{pk}, x)$:

1 : **parse** $\mathsf{crs} = (N, g, _)$
2 : **parse** $\mathsf{pk} = (f, _, _)$
3 : $r \leftarrow\!\$\ [2^{2\lambda}]$
4 : $\mathsf{ct}_0 \coloneqq (N+1)^x g^r \bmod N^{w+1}$
5 : $\mathsf{ct}_1 \coloneqq f^r \bmod N^{w+1}$
6 : $[\![x]\!]^\sigma_\sigma \coloneqq (r, \mathsf{ct}_0)$
7 : $[\![x]\!]^\sigma_{1-\sigma} \coloneqq (\mathsf{ct}_0, \mathsf{ct}_1)$
8 : **return** $([\![x]\!]^\sigma_A, [\![x]\!]^\sigma_B)$

$\mathsf{MKHSS.Eval}(\mathsf{crs}, \sigma, \mathsf{sk}_\sigma, \mathsf{pk}_{1-\sigma}, [\![\mathbf{x}_A]\!]^A_\sigma, [\![\mathbf{x}_B]\!]^B_\sigma, P)$:

1 : **parse** $\mathsf{crs} = (N, g, _)$
2 : **parse** $[\![\mathbf{x}_A]\!]^A_\sigma = [\![x^{(1)}_A]\!]^A_\sigma, \ldots, [\![x^{(m_A)}_A]\!]^A_\sigma$
3 : **parse** $[\![\mathbf{x}_B]\!]^B_\sigma = [\![x^{(1)}_B]\!]^B_\sigma, \ldots, [\![x^{(m_B)}_B]\!]^B_\sigma$
4 : $\mathsf{ek}_\sigma \coloneqq \mathsf{InitRMS}(\mathsf{crs}, \sigma, \mathsf{sk}_\sigma, \mathsf{pk}_{1-\sigma}, [\![\mathbf{x}_A]\!]^A_\sigma, [\![\mathbf{x}_B]\!]^B_\sigma)$
5 : **for** $i \in [m_\sigma]$, / Sync one's own shares
6 : $\quad \{\!\{x^{(i)}_\sigma\}\!\} \coloneqq \mathsf{ExpLinEncS}(\mathsf{sk}_\sigma, \mathsf{ek}_\sigma, [\![x^{(i)}_\sigma]\!]^\sigma_\sigma)$
7 : **for** $i \in [m_{1-\sigma}]$, / Sync received shares
8 : $\quad \{\!\{x^{(i)}_{1-\sigma}\}\!\} \coloneqq \mathsf{ExpLinEncR}(\mathsf{sk}_\sigma, \mathsf{pk}_{1-\sigma}, [\![x^{(i)}_{1-\sigma}]\!]^{1-\sigma}_\sigma)$
9 : $\{\!\{\mathbf{x}\}\!\} \coloneqq \{\!\{x^{(1)}_A\}\!\}, \ldots, \{\!\{x^{(m_A)}_A\}\!\}, \{\!\{x^{(1)}_B\}\!\}, \ldots, \{\!\{x^{(m_B)}_B\}\!\}$
10 : **return** $\mathsf{DEval}(\sigma, \mathsf{ek}_\sigma, \{\!\{\mathbf{x}\}\!\}, P)$

$\mathsf{DEval}(\sigma, \mathsf{ek}_\sigma, (\{\!\{x_1\}\!\}, \ldots, \{\!\{x_m\}\!\}), P)$:

Parse $\mathsf{ek}_\sigma = (k_{\mathsf{rms}}, \langle\!\langle 1 \rangle\!\rangle_\sigma, f)$.
For each $\mathsf{id} \in [|P|]$, evaluate the id-th instruction:

- $\mathsf{Convert}: \mathsf{M}_x \leftarrow \mathsf{I}_x$:
  1: Execute $\langle\!\langle x \rangle\!\rangle_\sigma \leftarrow \mathsf{Mult}(\{\!\{x\}\!\}, \langle\!\langle 1 \rangle\!\rangle_\sigma)$.
- $\mathsf{Mult}: \mathsf{M}_{xy} \leftarrow \mathsf{I}_x \cdot \mathsf{M}_y$:
  1: Parse $\{\!\{x\}\!\} = (\mathsf{ct}_0, \mathsf{ct}_1)$.
  2: Parse $\langle\!\langle y \rangle\!\rangle_\sigma = (\langle y \cdot s \rangle_\sigma, \langle y \rangle_\sigma)$.
  3: $\langle xys \rangle'_\sigma \leftarrow \mathsf{DDLog}_{N,w}(\mathsf{ct}_0^{\langle y \cdot s \rangle_\sigma} \mathsf{ct}_1^{\langle y \rangle_\sigma}; F_{N^w}(k_{\mathsf{rms}}, \mathsf{id}))$.
  4: $\langle xys \rangle_\sigma \leftarrow \langle xys \rangle'_\sigma \bmod B^3 \cdot 2^{4\tau+3\lambda}$.
  5: $\langle xy \rangle_\sigma \leftarrow \langle xys \rangle'_\sigma \bmod B \cdot 2^\tau$.
  6: $\langle\!\langle xy \rangle\!\rangle_\sigma \coloneqq (\langle xys \rangle_\sigma, \langle xy \rangle_\sigma)$
- $\mathsf{ScaleAdd_M}: \mathsf{M}_{ax+by} \leftarrow a\mathsf{M}_x + b\mathsf{M}_y$:
  1: $\langle\!\langle ax + by \rangle\!\rangle_\sigma \coloneqq a\langle\!\langle x \rangle\!\rangle_\sigma + b\langle\!\langle y \rangle\!\rangle_\sigma$.
- $\mathsf{ScaleAdd_I}: \mathsf{I}_{ax+by} \leftarrow a\mathsf{I}_x + b\mathsf{I}_y$:
  1: $\{\!\{ax + by\}\!\} \coloneqq \{\!\{x\}\!\}^a \cdot \{\!\{y\}\!\}^b$.
- $\mathsf{Output}: z \leftarrow \mathsf{M}_z$:
  1: Parse $\langle\!\langle z \rangle\!\rangle_\sigma = (\langle z \cdot s \rangle_\sigma, \langle z \rangle_\sigma)$.
  2: Return $\langle z \rangle_\sigma$.

$\mathsf{InitRMS}(\mathsf{crs}, \sigma, \mathsf{sk}_\sigma, \mathsf{pk}_{1-\sigma}, [\![\mathbf{x}_A]\!]^A_\sigma, [\![\mathbf{x}_B]\!]^B_\sigma)$:

1 : **parse** $\mathsf{sk}_\sigma = (s_\sigma, \mathsf{st}^{\mathsf{nim}})$
2 : **parse** $\mathsf{pk}_{1-\sigma} = (f_{1-\sigma}, \mathsf{pe}^{\mathsf{nim}}_A, \mathsf{pe}^{\mathsf{nim}}_B)$
3 : $f \coloneqq f_{1-\sigma}^{s_\sigma}$ / Precomputed f speeds up ExpLinEncS
4 : $k_{\mathsf{rms}} \| k_{\mathsf{nim}} \coloneqq H(f)$
5 : $\langle t_A t_B \rangle'_\sigma \coloneqq \mathsf{NIM.Decode}(\mathsf{crs}, \sigma, \mathsf{st}^{\mathsf{nim}}, \mathsf{pe}^{\mathsf{nim}}_{1-\sigma}; G_{N^w}(k_{\mathsf{nim}}))$
6 : $\langle t_A t_B \rangle_\sigma \coloneqq \langle t_A t_B \rangle'_\sigma \bmod 2^{4\lambda+\tau}$
7 : $\langle 1 \rangle_\sigma \coloneqq 1$ **if** $\sigma = A$ **else** 0
8 : $\langle t_A + t_B \rangle_\sigma \coloneqq t_A$ **if** $\sigma = A$ **else** $-t_B$
9 : $\langle s \rangle_\sigma \coloneqq \langle t_A t_B \rangle_\sigma (B')^2 + \langle t_A + t_B \rangle_\sigma B' + \langle 1 \rangle_\sigma$
10 : $\langle\!\langle 1 \rangle\!\rangle_\sigma \coloneqq (\langle s \rangle_\sigma, \langle 1 \rangle_\sigma)$
11 : $\mathsf{ek}_\sigma \coloneqq (k_{\mathsf{rms}}, \langle\!\langle 1 \rangle\!\rangle_\sigma, f)$
12 : **return** ek_σ

$\mathsf{ExpLinEncS}(\mathsf{sk}_\sigma, \mathsf{ek}_\sigma, [\![x^{(i)}_\sigma]\!]^\sigma_\sigma)$:

1 : **parse** $\mathsf{ek}_\sigma = (_, _, f)$
2 : **parse** $\mathsf{sk}_\sigma = (s_\sigma, _)$
3 : **parse** $[\![x^{(i)}_\sigma]\!]^\sigma_\sigma = (r, \mathsf{ct}_0)$
4 : $\{\!\{x^{(i)}_\sigma\}\!\} \coloneqq (\mathsf{ct}_0, f^r)$
5 : **return** $\{\!\{x^{(i)}_\sigma\}\!\}$

$\mathsf{ExpLinEncR}(\mathsf{sk}_\sigma, \mathsf{pk}_{1-\sigma}, [\![x^{(i)}_{1-\sigma}]\!]^{1-\sigma}_\sigma)$:

1 : **parse** $\mathsf{sk}_\sigma = (s_\sigma, _)$
2 : **parse** $\mathsf{pk}_{1-\sigma} = (f_{1-\sigma}, _, _)$
3 : **parse** $[\![x^{(i)}_{1-\sigma}]\!]^{1-\sigma}_\sigma = (\mathsf{ct}_0, \mathsf{ct}_1)$
4 : $\{\!\{x^{(i)}_{1-\sigma}\}\!\} \coloneqq (\mathsf{ct}_0, \mathsf{ct}_1^{s_\sigma})$
5 : **return** $\{\!\{x^{(i)}_{1-\sigma}\}\!\}$

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.

Appendix F.
Meta-Review

The following meta-review was prepared by the program committee for the 2026 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

F.1. Summary

This paper presents a concrete improvement to Multi-Key Homomorphic Secret Sharing (MKHSS) that enables homomorphic multiplication in just ≈ 5 ms. Then, it shows how to use MKHSS to enable attribute-based non-interactive key exchange (ANIKE). Finally, it demonstrates two applications of ANIKE: geolocation-based key exchange and password-based key exchange with fuzzy password comparison.

F.2. Scientific Contributions

- Creates a New Tool to Enable Future Science
- Provides a Valuable Step Forward in an Established Field

F.3. Reasons for Acceptance

1) The paper creates a new tool to enable future science. MKHSS is a new and active area of research, making MKHSS practical is an important problem, and practical improvements to MKHSS can be a step toward future efficient protocols.
2) The paper provides a valuable step forward in an established field. The proposed MKHSS scheme provides a significant performance improvement compared to the prior state of the art.

F.4. Noteworthy Concerns

1) Some aspects of the applications are not fully developed. For the geolocation-based key exchange application, one of the parties can fake their location.
2) Although the paper explains the benefits of non-interactivity in Section 1.1, it does not provide a quantitative performance comparison of the proposed MKHSS scheme with interactive approaches (e.g., secure multi-party computation).
3) The proposed MKHSS scheme is based on an existing protocol (Couteau et al., EUROCRYPT 2025) and the theoretical contributions seem somewhat limited. However, the reviewers felt that the practical developments were significant and sufficient for acceptance.

Authorized licensed use limited to: Anhui University. Downloaded on October 03,2026 at 01:26:28 UTC from IEEE Xplore. Restrictions apply.